

Table of Contents

Table of Contents.....	1
List of Figures	3
List of Tables	5
Abstract.....	6
1.0 Introduction.....	7
1.1 Background	7
1.2 Blockchain in Healthcare	8
1.3 Aim of the Project	9
1.4 Objectives	9
2.0 Literature Review and Theoretical Background	9
2.1 Blockchain Fundamentals	9
2.2 Cryptography in Blockchain	10
2.3 Consensus Mechanisms	10
2.4 Smart Contracts	10
2.5 Blockchain Applications in Healthcare	11
3.0 Blockchain Design and Implementation (Python).....	11
3.1 System Requirements	11
3.2 Block Structure	12
3.3 Genesis Block Creation	14
3.4 Proof-of-Work Mechanism	15
3.5 Patient Record Management and Block Creation	16
3.6 Blockchain Validation	17
3.7 Tampering Detection	17
3.8 Consensus Mechanism	18
3.9 Evaluation of the Python Blockchain Implementation	18
4.0 Smart Contract Design and Implementation.....	19
4.1 Chapter Introduction	19
4.2 Contract Overview	19
4.3 Data Structures and State Variables	19
4.4 Access Control and Security Mechanisms	21
4.5 Core Smart Contract Functions	22
4.6 Event Logging and Transparency	25
4.7 Security Considerations	26

4.8 Design Justification	27
5.0 Deployment and Ethereum Integration	27
5.1 Chapter Introduction	27
5.2 Ethereum Development Environment Configuration	27
5.3 Smart Contract Compilation	28
5.4 Deployment to the Ethereum Sepolia Test Network	29
5.5 Transaction Confirmation and Deployment Verification	30
5.6 Smart Contract Verification	31
5.7 Interaction with the Deployed Smart Contract	33
5.8 Evaluation of Deployment	33
6.0 Testing and Evaluation	34
6.1 Chapter Introduction	34
6.2 Testing Strategy	34
6.3 Python Blockchain Testing	35
6.4 Smart Contract Testing	36
6.5 Evaluation of Testing Results	41
7.0 Discussion and Critical Analysis	41
7.1 Chapter Introduction	41
7.2 Strengths of the Proposed Solution	42
7.3 Limitations of the Proposed Solution	42
7.4 Ethical and Legal Considerations	42
7.5 Recommendations for Future Development	43
8.0 Conclusion	43
References	45
Appendices	49
Appendix 1: Gantt Chart showing the timeline for project completion	49
Appendix 2: Development Environment Configuration	49
Appendix 3: Python Blockchain Source Code	50
Appendix 4: Solidity Smart Contract Source Code	50

List of Figures

Figure 3.1: Python Block Structure Implementation

Figure 3.2: Genesis Block Creation Output

Figure 3.3: Proof-of-Work Mining Output

Figure 3.4: Patient Record Transactions Stored within the Blockchain

Figure 3.5: Blockchain Validation Output

Figure 3.6: Tampering Detection Demonstration

Figure 3.7: Consensus Mechanism Simulation

Figure 4.1: PatientRecordContract Declaration

Figure 4.2: Patient Record Structure

Figure 4.3: Smart Contract State Variables

Figure 4.4: Access Control Modifiers

Figure 4.5: Provider Authorisation Function

Figure 4.6: Provider Revocation Function

Figure 4.7: Add Record Function

Figure 4.8: Transfer Record Function

Figure 4.9: Event Definitions

Figure 5.1: Alchemy Project Configuration and Sepolia Endpoint

Figure 5.2: Successful Smart Contract Compilation in Remix IDE

Figure 5.3: Remix IDE Connected to MetaMask and Sepolia Test Network

Figure 5.4: Successful Smart Contract Deployment

Figure 5.5: Deployment Transaction Confirmation

Figure 5.6: Successful Sourceify Verification

Figure 5.7: Successful Blockscout Verification

Figure 6.1: Authorise a Second Healthcare Provider

Figure 6.2: Adding New Record to the Smart Contract

Figure 6.3: Record Retrieval Function Output

Figure 6.4: Record Transfer Transaction

Figure 6.5: Patient History Retrieval Output

Figure A.1: Gantt Chart Showing the Eight-Week Project Schedule for the Development of a Blockchain-Based Patient Record Management System

Figure A.3: Complete Python implementation

Figure A.4: Complete PatientRecordContract.sol implementation

List of Tables

Table 3.1: Block Attributes

Table 4.1: Patient Record Structure Attributes

Table B.1: Development Environment Configuration

Abstract

Healthcare organisations are now experiencing several new problems as a result of poor safety, integrity, accessibility, and management of patient documentation. Traditional Electronic Health Record (EHR) systems are susceptible to data breaches, unauthorised changes and lack interoperability among systems. Due to its decentralised system, combined with the security of cryptography, immutability, and transparency, blockchain technology is now one of the best solutions to these problems.

The HealthCare Innovations Ltd. project, using the Python programming language and Ethereum smart contracts, focused on the design, construction, and evaluation of a new blockchain-based patient record management system. A custom blockchain, built using Python, incorporated the creation of blocks, the use of cryptographic hashes, the implementation of the Proof-of-Work consensus, and the validation of the blockchain, among other features, to demonstrate blockchain concepts. To move the solution outside a completely simulated environment, a Solidity smart contract, PatientRecordContract, was developed and deployed to the Ethereum Sepolia Test Network. The smart contract provided functionality to authorise healthcare providers, create patient records, transfer the ownership of patient records, retrieve patient record histories, and log related events.

The system was deployed to blockchain infrastructure using Remix IDE, MetaMask, and Alchemy, with contract validation performed via Sourceify and Blockscout. Extensive evaluations of the blockchain and smart contract systems, security and management of healthcare transactions were performed. The evaluations confirmed that the management of patient healthcare records using blockchain technology was effective.

The research established that blockchain technology, particularly in the realm of security, improves patient record management in healthcare significantly. Although a few major concerns, such as scalability, costs of transactions, privacy, and regulations, are yet to be considered and addressed for widespread adoption, these must be considered before large-scale implementations.

1.0 Introduction

Digital technologies have changed how healthcare information is developed, stored, organized, and consumed. Leading to the widespread implementation of Electronic Health Records (EHRs), digital technologies aid in clinical decision making, enhance the continuity of care, and promote operational efficiency within healthcare institutions. Unfortunately, traditional healthcare information systems are still hindered by security, interoperability, and data integrity issues, as well as unauthorized access and fragmented data storage (Agbo, Mahmoud and Eklund, 2019; Hasselgren et al., 2020). The aforementioned issues have compelled healthcare institutions to adopt new technologies in order to have secure, trustworthy, and transparent healthcare information management systems.

Blockchain technologies are becoming some of the first viable solutions to the aforementioned issues. Their decentralized structure, combination of cryptographic security, transparency, and immutability have created enough interest in various healthcare support services to warrant further investigation of their possible implementation (Angraal, Krumholz and Schulz, 2017; Haleem et al., 2021). Initially developed as the underlying technology for Bitcoin, blockchain has become a viable framework for virtually all other services spanning finance, supply chain management, government services, and healthcare (Di Francesco Maesa and Mori, 2020). Among the various services in healthcare, blockchain can facilitate security of patient data, ease interoperability, and enhance traceability, and increase trust of the various stakeholders in healthcare (El-Gazzar and Stendal, 2020).

This project was commissioned by HealthCare Innovations Ltd. to design and develop a blockchain patient record management system. The proposed system is a result of the combination of a custom, Python-based, client-side, and permissioned blockchain system and an Ethereum Sepolia Test Network Smart Contract. This project illustrates some of the possible enhancements of decentralized blockchain technologies in the healthcare industry with an emphasis on the security, transparency, traceability and integrity of blockchain healthcare records, as well as the provision of controlled access to sensitive healthcare information.

1.1 Background

Nakamoto (2008) first introduced Blockchain technology as the system for Bitcoin, then later as an advanced form of distributed ledger technology. This technology is now spanning many

more use cases in other industries (Angraal, Krumholz and Schulz, 2017; Di Francesco Maesa and Mori, 2020). Blockchain has the ability to create an unchangeable ledger that records transactions on a system that is verified by the users of the system. This ledger does not require a centralized body to manage it.

Healthcare organizations contain an increasing abundance of very sensitive data on patients. The digitization of health care with Electronic Health Record systems has improved the state of health care information systems, but many health care information systems are still centralized and brittle to system breaches (McGhin et al., 2019). Fragmented information systems are an obstacle to health care provider information exchange, which ultimately results in increased cost, duplication of tests, delays in treatment and an increased risk to the safety of the patient (Hussien et al., 2021). The sustained increase of cyberattacks on healthcare organizations indicates that the security of health care information systems is inadequate (Exceline and Nagarajan, 2023).

The introduction of Blockchain technology at HealthCare Innovations Ltd. aims to improve the management of patient records. According to Yaqoob et al. (2021), the use of decentralization, cryptographic hashing, consensus, and smart contracts offer health care organizations the ability to create secure, unchangeable systems that are more trustworthy and accountable, with higher quality data.

1.2 Blockchain in Healthcare

Agbo et al. (2019); Haleem et al. (2021); Hasselgren et al. (2020) assert that Blockchain technology has notable benefits for the management of health care records. The use of cryptographic hashing in Blockchain technology has the ability to secure medical records, thereby making them virtually unalterable. The use of Blockchain also promotes transparency due to the maintained comprehensive and auditable history of all transactions. Since all transactions previously in the Blockchain are secure cryptographically linked to the following transactions, unauthorized changes are evident. Moreover, decentralisation further decreases reliance on a singular database and decreases the probability of a system error.

From a healthcare point of view, decentralisation promotes the secure exchange of patient records as well as the provision of interoperable systems and clinical audits, and offers assurance to the various healthcare service providers (Vazirani et al., 2019). In addition, the

use of smart contracts aids in the further automation of access control and record management, decreases administrative burden, and ensures adherence to the rules (Khatoon, 2020).

1.3 Aim of the Project

The objective of this research is to design, build, and assess a blockchain-based patient record management system for HealthCare Innovations Ltd. using Python and Ethereum smart contracts.

1.4 Objectives

1. To design and implement a blockchain architecture using Python.
2. To develop a Proof-of-Work consensus mechanism for block validation.
3. To create functionality for patient record creation, storage, transfer, and verification.
4. To design and implement a Solidity smart contract for healthcare record management.
5. To deploy the smart contract on the Ethereum Sepolia Test Network.
6. To test and evaluate both the blockchain implementation and smart contract functionality.
7. To critically assess the benefits, limitations, and suitability of blockchain technology for healthcare applications.

2.0 Literature Review and Theoretical Background

2.1 Blockchain Fundamentals

Blockchain technology is a form of distributed ledger technology that makes the secure recording, validating, and storing of transactions on a decentralised network (Agbo et al., 2019; Zheng et al., 2020) possible. In contrast to a traditional database which employs a centralised system, blockchain technology provides a means of distributing the same ledger to multiple nodes.

A blockchain is made up of a series of linked lists, or blocks, which contain the recorded transactions, timestamps, hashes, and links to the previous blocks. Because of the way the technology is structured, blockchain technology provides a means of creating a tamper resistant system for the management of sensitive data (Bodkhe et al., 2020).

2.2 Cryptography in Blockchain

The foundation of the security of blockchain technology is the use of cryptography. In a blockchain system, the use of hash functions creates a fixed length output for input data, the hash value, which is unique and irreversible with respect to the input data. Thus, the system is cryptographically secure (Zheng et al., 2020; Bodkhe et al., 2020).

Since a blockchain system links the hash of the previous block to the current block, a tamper evident system is created. This method safeguards stored healthcare data from tampering and fosters confidence in distributed healthcare networks. (Patel, 2019).

2.3 Consensus Mechanisms

Consensus mechanisms allow participants in a distributed system to reach an agreement on the validity of transactions without a centralized authority (Saleh, 2020; Bodkhe et al., 2020). In the case of Proof-of-Work (PoW), participants in the network are required to solve very difficult computational problems to append a block to the ledger. While PoW has a very high guarantee of security, it is also very costly in terms of computation and energy.

Addressing the problems of scalability and sustainability has led to newer consensus mechanisms, such as Proof-of-Stake (PoS) (Saleh, 2020; Mearian, 2025).

2.4 Smart Contracts

A smart contract is a self-executing program with the terms of the agreement between buyer and seller in the form of code that runs on a blockchain network (Buterin, 2014; Wood, 2022; Khatoon, 2020). The Ethereum network made smart contracts more useful with the development of the Solidity programming language and a decentralized environment for their execution.

Smart contracts automate the function of a governing body and the administrative processes involved, and remove the need for third-party participants. Once they are deployed to the blockchain, they become reliable and accountable as they operate transparently and are immutable without permission (Zheng et al., 2020).

2.5 Blockchain Applications in Healthcare

Among emerging blockchain technology use cases, healthcare is the most disruptive (Vazirani et al., 2019; Tandon et al., 2020; Yaqoob et al., 2021). Blockchain provides a mechanism to make Electronic Health Record systems more secure, provide better interoperability, and give patients greater control over their health information (Mamun, Azam and Gritti, 2022).

Blockchain can make fraud more difficult and improve the integrity of data across the healthcare ecosystem. Other use cases include managing the pharmaceutical supply chains, regulatory compliance for clinical trials, managing patient consent, processing health insurance claims, and conducting medical research. (Siyal et al., 2019; Tith et al., 2020; Sethi et al., 2025). Blockchain systems also improve data validity, and encourage system wide trust, of healthcare delivery system networks, and optimize the availability of data.

Even with the advantages of blockchain systems, issues with scaling, privacy, and especially interoperability and integration, as well as the adoption and acceptance by different organizations, have been seen to hinder the implementation of blockchain technologies (McGhin et al., 2019; Saranya and Murugan, 2021). Thus, for successful integration and implementation, usage of blockchain technologies in healthcare systems needs a detailed and well thought out technical and regulatory framework.

3.0 Blockchain Design and Implementation (Python)

This chapter describes the design and implementation of a blockchain-based patient record management system for HealthCare Innovations Ltd. It was built using Python to provide a working example of fundamental blockchain features, such as creating a block, cryptographic hash, Proof-of-Work, consensus, validation of the chain, detection of tampering, and consensus of distribution. The prototype provides a framework to manage patient records within a safe and transparent system, and fosters trust, traceability, and data integrity of healthcare record management systems (Shahnaz, Qamar and Khalid, 2019; Omar et al., 2019).

3.1 System Requirements

Fragmentation of data storage, limited transparency, and the ability to modify data are challenges of conventional healthcare information systems that this blockchain solution is

designed to overcome (Patel, 2019). The solution had to exhibit the core features of blockchain, while facilitating exchange of healthcare-related information.

The key functional requirements included:

- Creation of a genesis block.
- Addition of new blocks containing healthcare transactions.
- Implementation of a Proof-of-Work mining mechanism.
- Secure storage of patient record information.
- Blockchain validation and integrity checking.
- Detection of data tampering attempts.
- Simulation of distributed consensus using multiple nodes.

The focus of the healthcare use case was the creation and transfer of patient records among healthcare providers. This use case also made it possible to demonstrate the blockchain concepts in a healthcare environment that is as close to the actual scenario as possible.

3.2 Block Structure

Central to the blockchain system was the Block class. The integrity of the chain and the continuity of the ledger was maintained by the transactions stored in the blocks.

```

jupyter LDS7006M_Blockchain_Python Last Checkpoint: yesterday
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

Cell 1 — Imports and Dependencies
[3]: import hashlib
import time
import json
import copy

print("Libraries loaded successfully.")

Libraries loaded successfully.

Cell 2 — Block Class Structure
[4]: class Block:
    """
    Represents a single block in the blockchain.
    Each block stores: index, previous hash, timestamp, data, nonce, and its own hash.
    """

    def __init__(self, index, previous_hash, timestamp, data, nonce=0):
        self.index = index
        self.previous_hash = previous_hash
        self.timestamp = timestamp
        self.data = data
        self.nonce = nonce
        self.hash = self.compute_hash()

    def compute_hash(self):
        """Serialises block fields and returns a SHA-256 hash string."""
        block_content = json.dumps({
            "index": self.index,
            "previous_hash": self.previous_hash,
            "timestamp": self.timestamp,
            "data": self.data,
            "nonce": self.nonce
        }, sort_keys=True)
        return hashlib.sha256(block_content.encode()).hexdigest()

    def __repr__(self):
        return (
            f"Block(index={self.index}, "
            f"hash={self.hash[:12]}..., "
            f"preve={self.previous_hash[:12]}..., "
            f"nonce={self.nonce})"
        )

# — Quick test —
test_block = Block(
    index=0,
    previous_hash="0" * 64,
    timestamp=time.time(),
    data="Test block"
)

print("Block created:")
print(f"Index : {test_block.index}")
print(f>Data : {test_block.data}")
print(f"Hash : {test_block.hash}")
print(f"Previous Hash : {test_block.previous_hash}")

Block created:
Index : 0
Data : Test block
Hash : 7e4a6cc5b8f2696db8b480460807598d9dd2c5502fc31dd2aa8572e54a9ef1b4
Previous Hash : 0000000000000000000000000000000000000000000000000000000000000000

```

Figure 3.1: Python Block Structure Implementation

The block structure consists of six primary attributes, as shown in Table 3.1.

Table 3.1: Block Attributes

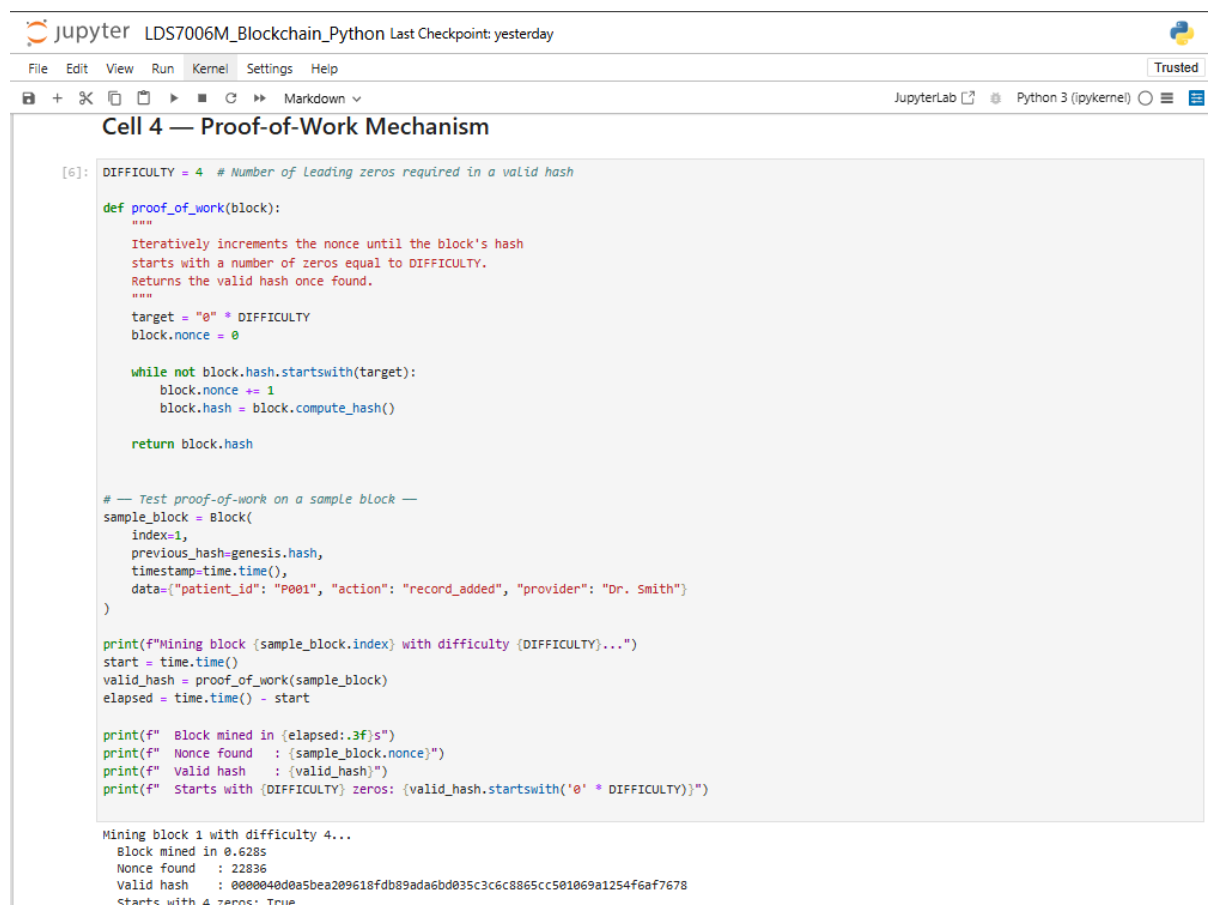
Attribute	Description

The genesis block was automatically created during the initialization of the blockchain. The previous hash was set to sixty-four zeros indicating that no block was created before it.

The successful creation of the genesis block confirmed that the blockchain was successfully initialized. It also created the first link to all following medical transactions.

3.4 Proof-of-Work Mechanism

The Proof-of-Work (PoW) consensus mechanism was designed to secure the blockchain and control the addition of new blocks (Saleh, 2020). To mine a block, the required hash had to meet a specific target of difficulty.



```
[6]: DIFFICULTY = 4 # Number of Leading zeros required in a valid hash

def proof_of_work(block):
    """
    Iteratively increments the nonce until the block's hash
    starts with a number of zeros equal to DIFFICULTY.
    Returns the valid hash once found.
    """
    target = "0" * DIFFICULTY
    block.nonce = 0

    while not block.hash.startswith(target):
        block.nonce += 1
        block.hash = block.compute_hash()

    return block.hash

# — Test proof-of-work on a sample block —
sample_block = Block(
    index=1,
    previous_hash=genesis.hash,
    timestamp=time.time(),
    data={"patient_id": "P001", "action": "record_added", "provider": "Dr. Smith"}
)

print(f"Mining block {sample_block.index} with difficulty {DIFFICULTY}...")
start = time.time()
valid_hash = proof_of_work(sample_block)
elapsed = time.time() - start

print(f" Block mined in {elapsed:.3f}s")
print(f" Nonce found   : {sample_block.nonce}")
print(f" Valid hash    : {valid_hash}")
print(f" Starts with {DIFFICULTY} zeros: {valid_hash.startswith('0' * DIFFICULTY)}")

Mining block 1 with difficulty 4...
Block mined in 0.628s
Nonce found   : 22836
Valid hash    : 0000040d0a5bea209618fdb89ada6bd035c3c6c8865cc501069a1254f6af7678
Starts with 4 zeros: True
```

Figure 3.3: Proof-of-Work Mining Output

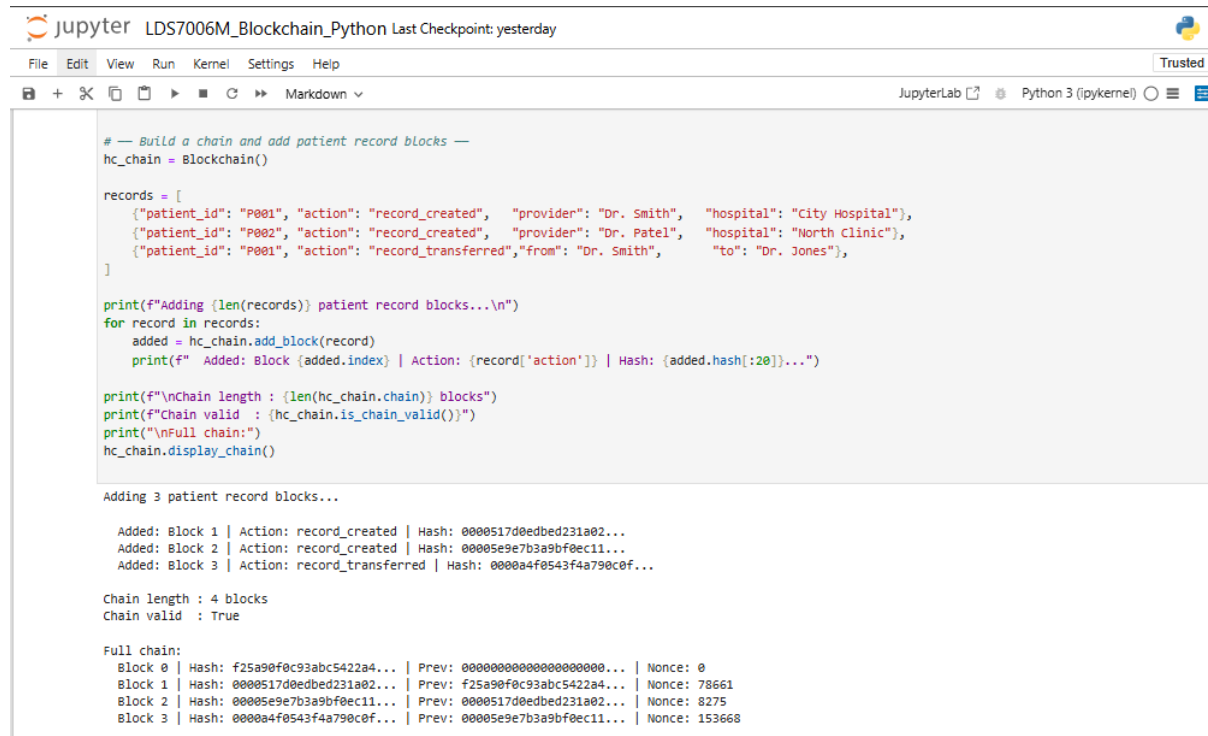
The target of the difficulty level was set to four leading zeros. The mining algorithm was allowed to change the nonce value until the target was met.

The successful mining of the block validated the claim that only blocks which met the target of difficulty could be added to the blockchain. Trust and security of healthcare information

from unauthorized modification was enhanced by the increased level of computational effort to change records (Bodkhe et al., 2020).

3.5 Patient Record Management and Block Creation

After the mining process was successfully established, the blockchain began the management of healthcare transactions.



```
# -- Build a chain and add patient record blocks --
hc_chain = Blockchain()

records = [
    {"patient_id": "P001", "action": "record_created", "provider": "Dr. Smith", "hospital": "City Hospital"},
    {"patient_id": "P002", "action": "record_created", "provider": "Dr. Patel", "hospital": "North Clinic"},
    {"patient_id": "P001", "action": "record_transferred", "from": "Dr. Smith", "to": "Dr. Jones"},
]

print(f"Adding {len(records)} patient record blocks...\n")
for record in records:
    added = hc_chain.add_block(record)
    print(f"Added: Block {added.index} | Action: {record['action']} | Hash: {added.hash[:20]}...")

print(f"\nChain length : {len(hc_chain.chain)} blocks")
print(f"Chain valid : {hc_chain.is_chain_valid()}")
print("\nFull chain:")
hc_chain.display_chain()

Adding 3 patient record blocks...

Added: Block 1 | Action: record_created | Hash: 0000517d0edbed231a02...
Added: Block 2 | Action: record_created | Hash: 00005e9e7b3a9bf0ec11...
Added: Block 3 | Action: record_transferred | Hash: 0000a4f0543f4a790c0f...

Chain length : 4 blocks
Chain valid : True

Full chain:
Block 0 | Hash: f25a90f0c93abc5422a4... | Prev: 00000000000000000000... | Nonce: 0
Block 1 | Hash: 0000517d0edbed231a02... | Prev: f25a90f0c93abc5422a4... | Nonce: 78661
Block 2 | Hash: 00005e9e7b3a9bf0ec11... | Prev: 0000517d0edbed231a02... | Nonce: 8275
Block 3 | Hash: 0000a4f0543f4a790c0f... | Prev: 00005e9e7b3a9bf0ec11... | Nonce: 153668
```

Figure 3.4: Patient Record Transactions Stored within the Blockchain

The system enabled the creation and transfer of patient records. For example, records were created for patient P001 and patient P002, and ownership was transferred between health care providers.

The process involved:

1. Creation of healthcare transaction data.
2. Generation of a new block.
3. Assignment of the previous block hash.
4. Execution of the Proof-of-Work algorithm.

5. Block validation.
6. Addition to the blockchain.

This method ensures that both a thorough record of healthcare transactions is maintained and that there is a permanent chain of custody established with cryptographic hashing (Patel, 2019).

3.6 Blockchain Validation

In a healthcare setting where data reliability is paramount to patient care, maintaining the integrity of blockchain is essential.

Figure 3.5: Blockchain Validation Output

The validation process performs two checks:

- Verification of each block hash.
- Verification of correct previous-hash references.

It was verified that all blocks were properly connected and valid. This mechanism ensures the security of health records and ensures that they are not altered (Shahnaz, Qamar and Khalid, 2019).

3.7 Tampering Detection

A tampering simulation was carried out to illustrate blockchain security.



```
jupyter LDS7006M_Blockchain_Python Last Checkpoint: yesterday
File Edit View Run Kernel Settings Help
JupyterLab Python 3 (ipykernel)
Block 4 mined | Nonce: 139424 | Action: record_created

[12]: # 3. Validate chain integrity
print(f"[3] Chain integrity check: {'PASSED' if demo_chain.is_chain_valid() else 'FAILED'}")

[3] Chain integrity check: PASSED

[13]: # 4. Tamper with a block and re-validate
print("\n[4] Simulating tampering with Block 1 data...")
demo_chain.chain[1].data = {"patient_id": "P999", "action": "TAMPERED"}
print(f"Chain integrity after tampering: {'PASSED' if demo_chain.is_chain_valid() else 'FAILED - Tampering detected'}")

[4] Simulating tampering with Block 1 data...
[INVALID] Block 1 hash mismatch.
Chain integrity after tampering: FAILED - Tampering detected
```

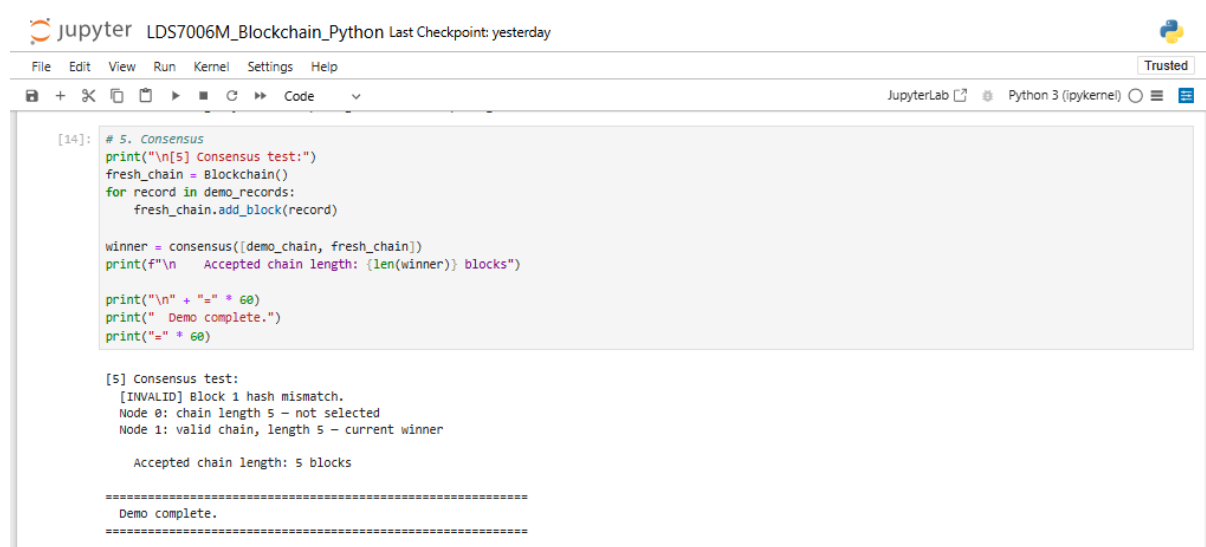
Figure 3.6: Integrity Check and Tampering Detection Demonstration

The data of a transaction in an existing block was intentionally changed prior to re-executing the validation algorithm. The system instantly identified a hash mismatch and then reported that the block chain was not valid.

The results of this experiment showcase the powers of the blockchain immutability and cryptographic protection mechanisms to shield healthcare information (Omar et al., 2019).

3.8 Consensus Mechanism

A simplified distributed consensus mechanism with the longest chain rule was implemented.



```
[14]: # 5. Consensus
print("\n[5] Consensus test:")
fresh_chain = Blockchain()
for record in demo_records:
    fresh_chain.add_block(record)

winner = consensus([demo_chain, fresh_chain])
print(f"\n    Accepted chain length: {len(winner)} blocks")

print("\n" + "=" * 60)
print(" Demo complete.")
print("=" * 60)

[5] Consensus test:
[INVALID] Block 1 hash mismatch.
Node 0: chain length 5 - not selected
Node 1: valid chain, length 5 - current winner

    Accepted chain length: 5 blocks

=====
 Demo complete.
=====
```

Figure 3.7: Consensus Mechanism Simulation

There were three separate nodes, each with varying lengths of the blockchain. The algorithm chose the longest valid chain as the 'authoritative version' of the ledger.

This behaviour is representative of the working of decentralised blockchain networks, where the participants agree on the state of the ledger without relying on a central authority (McGhin et al., 2019).

3.9 Evaluation of the Python Blockchain Implementation

The Python implementation of the blockchain worked well, showcasing block generation, healthcare transaction storage, Proof-of-Work mining, validation, tamper detection, and distributed consensus.

A number of strengths were found. Firstly, cryptographic hashing gave the immutability and data integrity. Secondly, Proof-of-Work increased security with a feature that thwarted

unauthorized block creation. Thirdly, validation and consensus mechanisms demonstrated decentralised trust management.

However, limitations exist. The prototype operates in a simulated environment without real peer-to-peer networking, and the healthcare records used for testing were simplified representations of clinical information. Consequently, additional development would be required before deployment in a production healthcare environment (Mamun, Azam and Gritti, 2022).

4.0 Smart Contract Design and Implementation

4.1 Chapter Introduction

While the Python implementation demonstrated fundamental blockchain concepts, smart contracts enable these concepts to operate within a live blockchain environment. This chapter discusses the design and implementation of the smart contract created for HealthCare Innovations Ltd. (HCIL).

4.2 Contract Overview

The smart contract, called **PatientRecordContract**, was implemented in Solidity and deployed onto the Ethereum Sepolia Test Network.

It is mainly designed to give a decentralized way to manage healthcare record metadata, with access control enforced via programmable business rules (Buterin, 2014; Wood, 2022).

Figure 4.1: PatientRecordContract Declaration

The contract allows for authorised healthcare providers to create, transfer and retrieve patient records, and keep a complete and unchangeable history of healthcare transactions.

4.3 Data Structures and State Variables

The contract specifies several state variables and data structures to help with patient record management.

Patient Record Structure

A Solidity struct was implemented to represent healthcare records.

```

33
34 // -----
35 // Structs
36 // -----
37
38 /**
39  * @dev Stores all details related to a single patient record entry.
40  *      recordHash is a SHA-256 hash of the full off-chain clinical data,
41  *      used to verify data integrity without exposing sensitive information.
42  */
43 struct PatientRecord {
44     uint256 recordId; // Unique identifier for this record
45     string patientId; // Hospital patient reference number
46     string providerName; // Full name of the healthcare provider
47     string hospital; // Name of the hospital or clinic
48     string diagnosisCode; // ICD-10 diagnosis code
49     bytes32 recordHash; // SHA-256 hash of the full off-chain clinical data
50     address currentProvider; // Ethereum address of the current record holder
51     uint256 timestamp; // Unix timestamp of when the record was created
52     bool isActive; // Whether the record is currently active
53 }
54
55

```

Figure 4.2: Patient Record Structure

The structure contains:

Attribute	Description
Record ID	Unique record identifier
Patient ID	Patient identifier
Record Hash	SHA-256 hash of patient information
Current Owner	Responsible healthcare provider
Timestamp	Creation timestamp
Active Status	Record status

Storing hashes, rather than full patient records, increases privacy and decreases the storage requirements on the blockchain (Tith et al., 2020).

State Variables

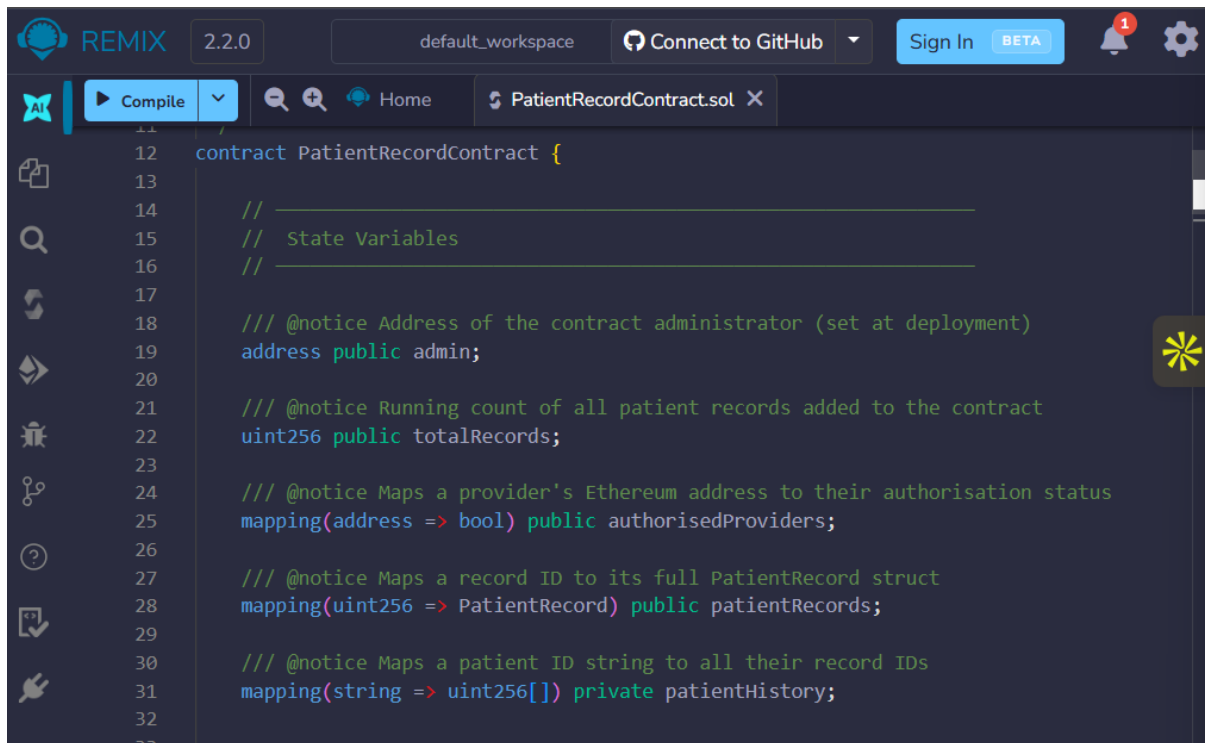


Figure 4.3: Smart Contract State Variables

Key variables include:

- admin
- totalRecords
- records
- patientHistory
- authorisedProviders

Mappings enable easy access to health care records and authorised provider details.

4.4 Access Control and Security Mechanisms

Healthcare data requires strict access-control mechanisms (Exceline and Nagarajan, 2023).

```
80 event ProviderRevoked(address indexed provider, uint256 timestamp);
81
82
83 //
84 // Modifiers
85 //
86
87 /**
88  * @dev Restricts a function so only the admin address can call it.
89  * Reverts with a descriptive message if the caller is not the admin.
90  */
91 modifier onlyAdmin() {
92     require(
93         msg.sender == admin,
94         "PatientRecordContract: caller is not the admin"
95     );
96     _;
97 }
98
99 /**
100  * @dev Restricts a function so only addresses in the authorisedProviders
101  * mapping can call it. Reverts if the caller is not authorised.
102  */
103 modifier onlyAuthorised() {
104     require(
105         authorisedProviders[msg.sender],
106         "PatientRecordContract: caller is not an authorised provider"
107     );
108     _;
109 }
110
111 /**
112  * @dev Validates that a given record ID exists and is active before
113  * allowing a function to proceed.
114  */
115 modifier recordExists(uint256 _recordId) {
116     require(
117         _recordId > 0 && _recordId <= totalRecords,
118         "PatientRecordContract: record ID does not exist"
119     );
120     require(
121         patientRecords[_recordId].isActive,
122         "PatientRecordContract: record is no longer active"
123     );
124     _;
125 }
```

Figure 4.4: Access Control Modifiers

Three Solidity modifiers were implemented:

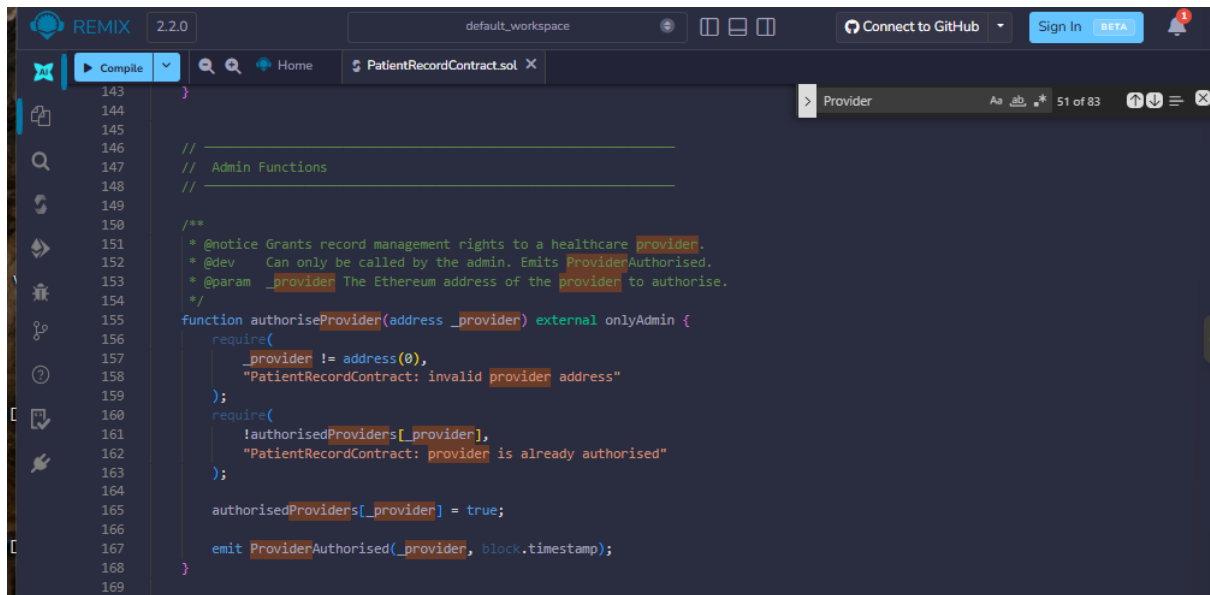
- **onlyAdmin:** Restricts access to administrative functions.
- **onlyAuthorised:** Ensures only approved providers can create or transfer records.
- **recordExists:** Verifies the existence of records before allowing access.

These mechanisms improve security and reduce the risk of unauthorised interaction with healthcare information.

4.5 Core Smart Contract Functions

Several functions were implemented to support healthcare record management:

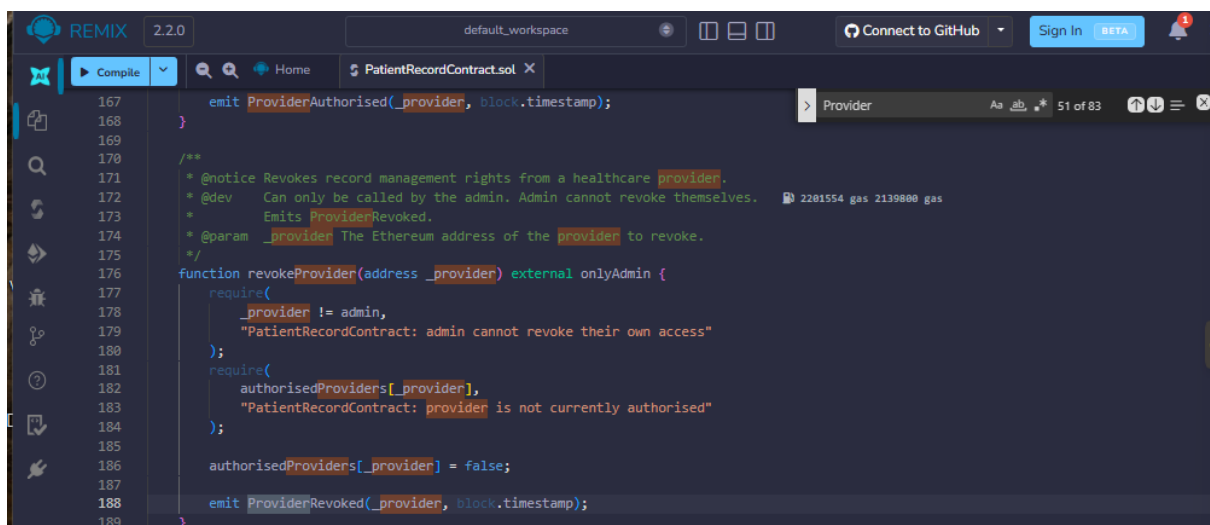
Authorise Provider: Allows administrators to approve healthcare providers.



```
143 }
144
145 //
146 // Admin Functions
147 //
148 //
149
150 /**
151  * @notice Grants record management rights to a healthcare provider.
152  * @dev Can only be called by the admin. Emits ProviderAuthorised.
153  * @param _provider The Ethereum address of the provider to authorise.
154  */
155 function authoriseProvider(address _provider) external onlyAdmin {
156     require(
157         _provider != address(0),
158         "PatientRecordContract: invalid provider address"
159     );
160     require(
161         !authorisedProviders[_provider],
162         "PatientRecordContract: provider is already authorised"
163     );
164     authorisedProviders[_provider] = true;
165     emit ProviderAuthorised(_provider, block.timestamp);
166 }
167
168
169
```

Figure 4.5: Provider Authorisation Function

Revoke Provider: Removes provider access privileges.



```
167     emit ProviderAuthorised(_provider, block.timestamp);
168 }
169
170 /**
171  * @notice Revokes record management rights from a healthcare provider.
172  * @dev Can only be called by the admin. Admin cannot revoke themselves.
173  * @dev Emits ProviderRevoked.
174  * @param _provider The Ethereum address of the provider to revoke.
175  */
176 function revokeProvider(address _provider) external onlyAdmin {
177     require(
178         _provider != admin,
179         "PatientRecordContract: admin cannot revoke their own access"
180     );
181     require(
182         authorisedProviders[_provider],
183         "PatientRecordContract: provider is not currently authorised"
184     );
185     authorisedProviders[_provider] = false;
186     emit ProviderRevoked(_provider, block.timestamp);
187 }
188
189
```

Figure 4.6: Provider Revocation Function

Add Record: Creates a new healthcare record by generating a unique identifier, assigning ownership, and recording the transaction on-chain.

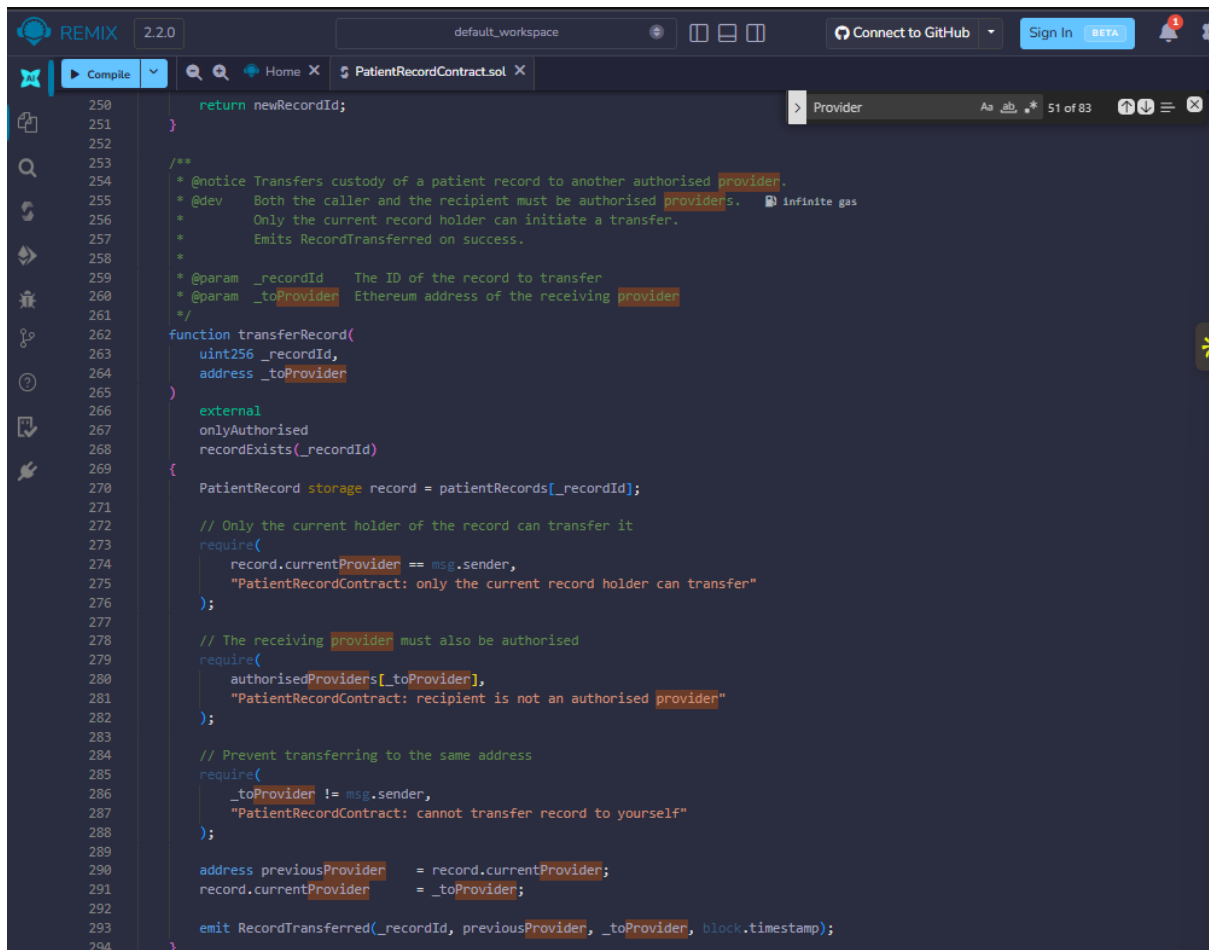
```

195
196
197 /**
198  * @notice Adds a new patient record to the blockchain.
199  * @dev Only authorised providers may call this function. 30863 gas
200  * totalRecords is incremented before assignment to
201  * ensure record IDs start at 1 (never 0).
202  * Emits RecordAdded on success.
203  *
204  * @param _patientId Hospital patient reference number (e.g. "P001")
205  * @param _providerName Full name of the adding provider (e.g. "Dr. Smith")
206  * @param _hospital Name of the hospital or clinic
207  * @param _diagnosisCode ICD-10 diagnosis code (e.g. "J18.9")
208  * @param _recordHash SHA-256 hash of the full off-chain clinical document
209  *
210  * @return newRecordId The ID assigned to the newly created record
211  */
212 function addRecord(
213     string memory _patientId,
214     string memory _providerName, 33016 gas
215     string memory _hospital,
216     string memory _diagnosisCode,
217     bytes32 _recordHash
218 )
219     external
220     onlyAuthorised
221     returns (uint256 newRecordId)
222 {
223     // Validate that no required string fields are empty
224     require(bytes(_patientId).length > 0, "PatientRecordContract: patient ID required");
225     require(bytes(_providerName).length > 0, "PatientRecordContract: provider name required");
226     require(bytes(_hospital).length > 0, "PatientRecordContract: hospital name required");
227     require(bytes(_diagnosisCode).length > 0, "PatientRecordContract: diagnosis code required");
228
229     // Increment counter first so IDs begin at 1
230     totalRecords++;
231     newRecordId = totalRecords;
232
233     // Build and store the new record
234     patientRecords[newRecordId] = PatientRecord({
235         recordId      : newRecordId,
236         patientId     : _patientId,
237         providerName  : _providerName,
238         hospital      : _hospital,
239         diagnosisCode : _diagnosisCode,
240         recordHash    : _recordHash,
241         currentProvider : msg.sender,
242         timestamp     : block.timestamp
243     });
244     emit RecordAdded(newRecordId, _patientId, _providerName, _hospital, _diagnosisCode, _recordHash);
245 }

```

Figure 4.7: Add Record Function

Transfer Record: Enables ownership transfer between authorised healthcare providers.



```
250     return newRecordId;
251 }
252
253 /**
254  * @notice Transfers custody of a patient record to another authorised provider.
255  * @dev Both the caller and the recipient must be authorised providers. Infinite gas
256  * Only the current record holder can initiate a transfer.
257  * Emits RecordTransferred on success.
258  *
259  * @param _recordId The ID of the record to transfer
260  * @param _toProvider Ethereum address of the receiving provider
261  */
262 function transferRecord(
263     uint256 _recordId,
264     address _toProvider
265 )
266 external
267 onlyAuthorised
268 recordExists(_recordId)
269 {
270     PatientRecord storage record = patientRecords[_recordId];
271
272     // Only the current holder of the record can transfer it
273     require(
274         record.currentProvider == msg.sender,
275         "PatientRecordContract: only the current record holder can transfer"
276     );
277
278     // The receiving provider must also be authorised
279     require(
280         authorisedProviders[_toProvider],
281         "PatientRecordContract: recipient is not an authorised provider"
282     );
283
284     // Prevent transferring to the same address
285     require(
286         _toProvider != msg.sender,
287         "PatientRecordContract: cannot transfer record to yourself"
288     );
289
290     address previousProvider = record.currentProvider;
291     record.currentProvider = _toProvider;
292
293     emit RecordTransferred(_recordId, previousProvider, _toProvider, block.timestamp);
294 }
```

Figure 4.8: Transfer Record Function

Retrieve Record: Supports retrieval of stored healthcare record information.

Retrieve Patient History: Returns all records associated with a specific patient.

These functions collectively support secure and transparent healthcare record management (Khatoon, 2020).

4.6 Event Logging and Transparency

Blockchain transparency was enhanced through Solidity events.

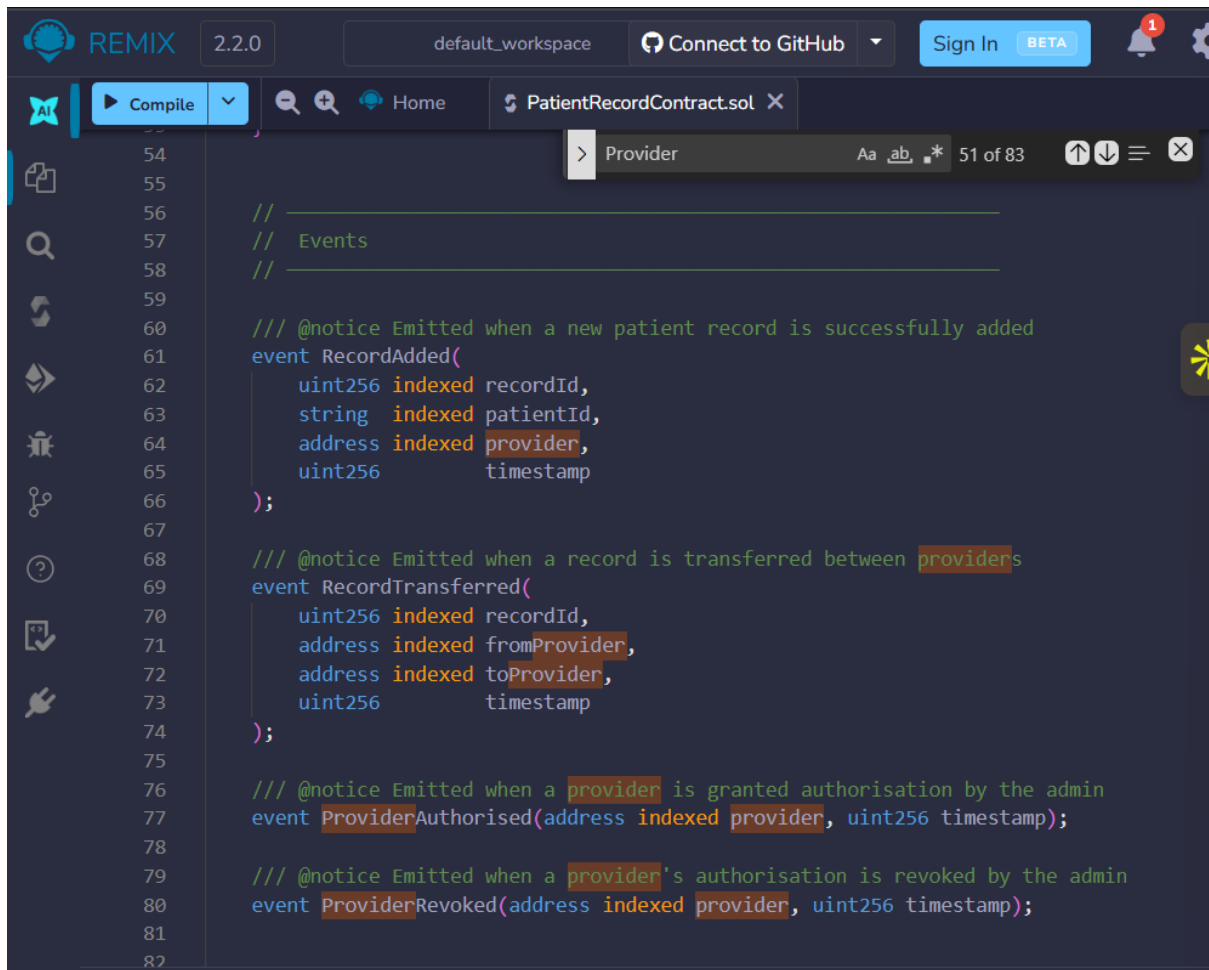


Figure 4.9: Event Definitions

Events were generated for:

- Record creation
- Record transfer
- Provider authorisation
- Provider revocation

Event logs create permanent and auditable records that can be independently verified through blockchain explorers (Ethereum Foundation, 2023).

4.7 Security Considerations

Security was a primary design objective.

Implemented measures included:

- Role-based access control.

- Input validation through require () statements.
- Record existence verification.
- Cryptographic hash storage instead of raw patient information.

These controls strengthen privacy protection and reduce the likelihood of misuse (Excelsior and Nagarajan, 2023; Tith et al., 2020).

4.8 Design Justification

Smart contracts have a lot of benefits over traditional healthcare information systems. They automate administrative processes, decrease dependence on intermediaries and guarantee consistent rule enforcement (Khatoon, 2020).

In healthcare settings, automated access control, immutable audit trails, and clear governance can enhance accountability and streamline operations. The prototype successfully shows the potential of smart contracts on Ethereum in the healthcare record management system, albeit with scope for further scalability and privacy measures in large-scale deployments.

5.0 Deployment and Ethereum Integration

5.1 Chapter Introduction

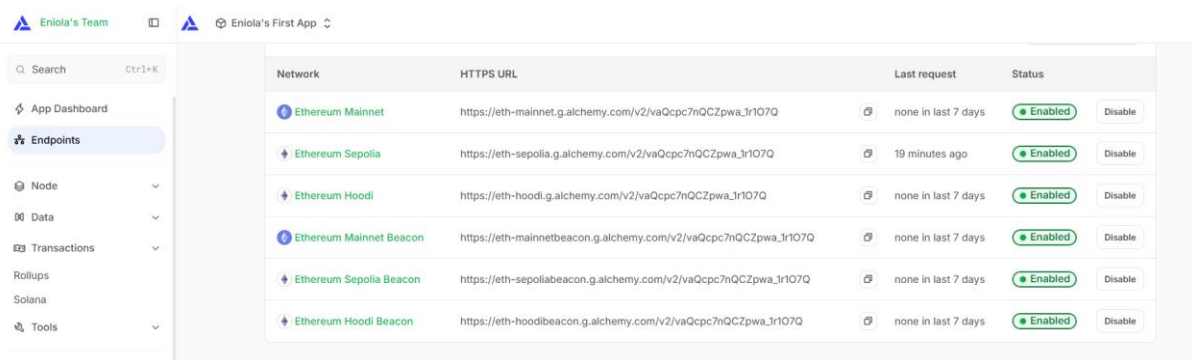
After the successful development of the PatientRecordContract smart contract, the next step was to deploy the smart contract to a real blockchain environment. This chapter outlines the deployment environment setup, compiling, deploying to the Ethereum Sepolia Test Network, verifying the contract, and interacting with the deployed contract.

5.2 Ethereum Development Environment Configuration

An Ethereum development environment was set up before deployment, and includes Remix IDE, MetaMask, Alchemy and the Ethereum Sepolia Test Network.

The reason for choosing Remix IDE is that it has the integrated tools for Solidity programming, debugging, compiling, deployment, and interacting with the contracts (Ethereum Foundation, 2023). The wallet consisted of MetaMask, a wallet management system, and transaction

signing, while blockchain infrastructure and Remote Procedure Call (RPC) connectivity were provided by Alchemy.



The screenshot shows the Alchemy Project Configuration interface. On the left is a sidebar with navigation options: App Dashboard, Endpoints (selected), Node, Data, Transactions, Rollups, Solana, and Tools. The main area displays a table of configured endpoints.

Network	HTTPS URL	Last request	Status
Ethereum Mainnet	https://eth-mainnet.g.alchemy.com/v2/vaQcpc7nQCZpwa_1r107Q	none in last 7 days	Enabled
Ethereum Sepolia	https://eth-sepolia.g.alchemy.com/v2/vaQcpc7nQCZpwa_1r107Q	19 minutes ago	Enabled
Ethereum Hoodi	https://eth-hoodi.g.alchemy.com/v2/vaQcpc7nQCZpwa_1r107Q	none in last 7 days	Enabled
Ethereum Mainnet Beacon	https://eth-mainnetbeacon.g.alchemy.com/v2/vaQcpc7nQCZpwa_1r107Q	none in last 7 days	Enabled
Ethereum Sepolia Beacon	https://eth-sepoliabeacon.g.alchemy.com/v2/vaQcpc7nQCZpwa_1r107Q	none in last 7 days	Enabled
Ethereum Hoodi Beacon	https://eth-hoodibeacon.g.alchemy.com/v2/vaQcpc7nQCZpwa_1r107Q	none in last 7 days	Enabled

Figure 5.1: Alchemy Project Configuration and Sepolia Endpoint

The Sepolia endpoint was then linked with MetaMask, which allowed for the deployment transactions to be securely sent to the Ethereum blockchain.

This is necessary since it enables developers to test smart contract functionality without risks to production systems and without the financial burden of real cost to the developer (Mearian, 2025).

5.3 Smart Contract Compilation

The PatientRecordContract was compiled beforehand using the Remix IDE's built-in Solidity compiler.

You should see the following screen if your Smart Contract has been compiled successfully. The following screen should appear if your Smart Contract was compiled successfully.

Compilation translates Solidity source code to Ethereum Virtual Machine (EVM) bytecode which then can be run by blockchain nodes (Wood, 2022). The process was successful and will confirm that all the functions in the contract, all state variables, modifiers and all event definitions were correctly implemented.

Compilation generated two important outputs:

- Application Binary Interface (ABI)
- Ethereum Virtual Machine Bytecode

The ABI describes the external applications' interaction with the contract and the bytecode is the executable version that is deployed to the blockchain.

5.4 Deployment to the Ethereum Sepolia Test Network

Following successful compilation, the PatientRecordContract was deployed through Remix IDE using MetaMask.

Deployment configuration included:

- Environment: Injected Provider – MetaMask
- Network: Ethereum Sepolia Test Network
- Deployment Platform: Remix IDE

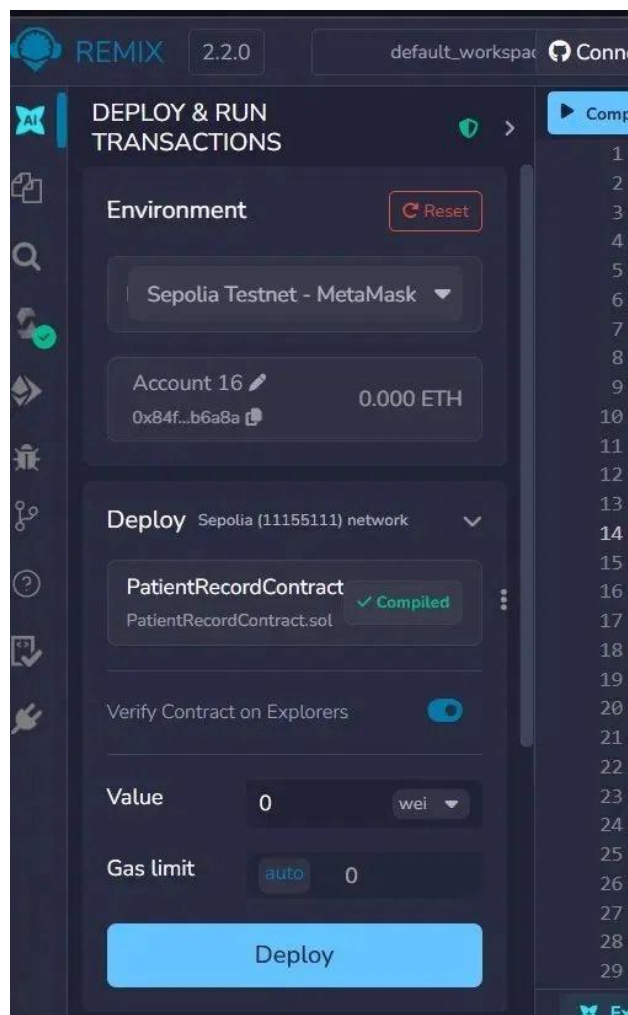


Figure 5.3: Remix IDE Connected to MetaMask and Sepolia Test Network

The deployment transaction was signed using MetaMask and sent to the Sepolia network, where it was approved and added to the blockchain.

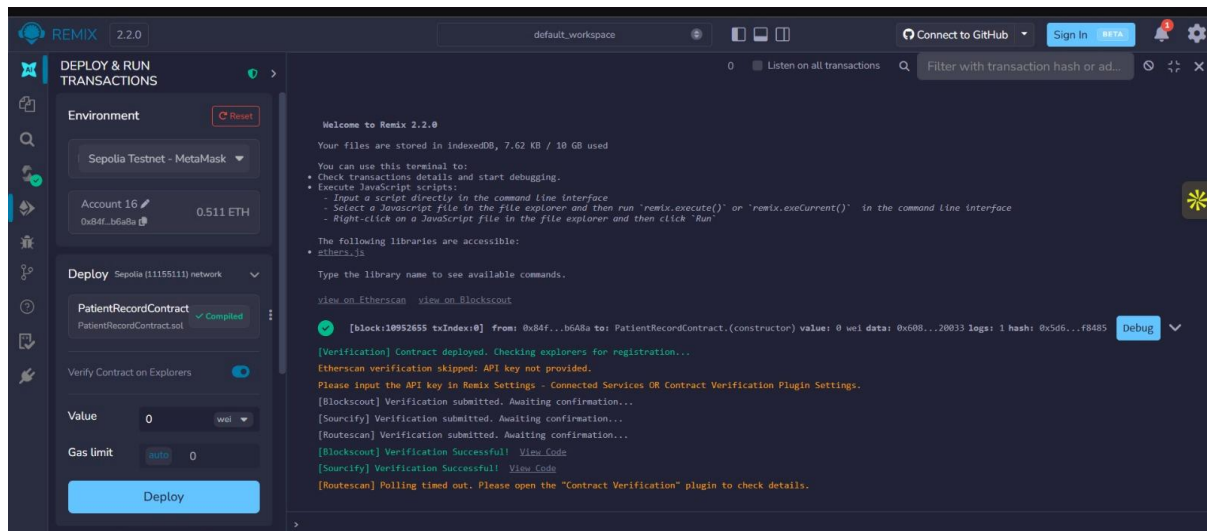


Figure 5.4: Successful Smart Contract Deployment

The successful deployment also proved the compatibility of the Ethereum infrastructure and the feasibility of the suggested healthcare solution (Buterin, 2014).

5.5 Transaction Confirmation and Deployment Verification

Following deployment, confirmation information was generated, including:

- Contract Creator Address
- Contract Address
- Transaction Hash
- Block Number
- Gas Consumption
- Confirmation Status

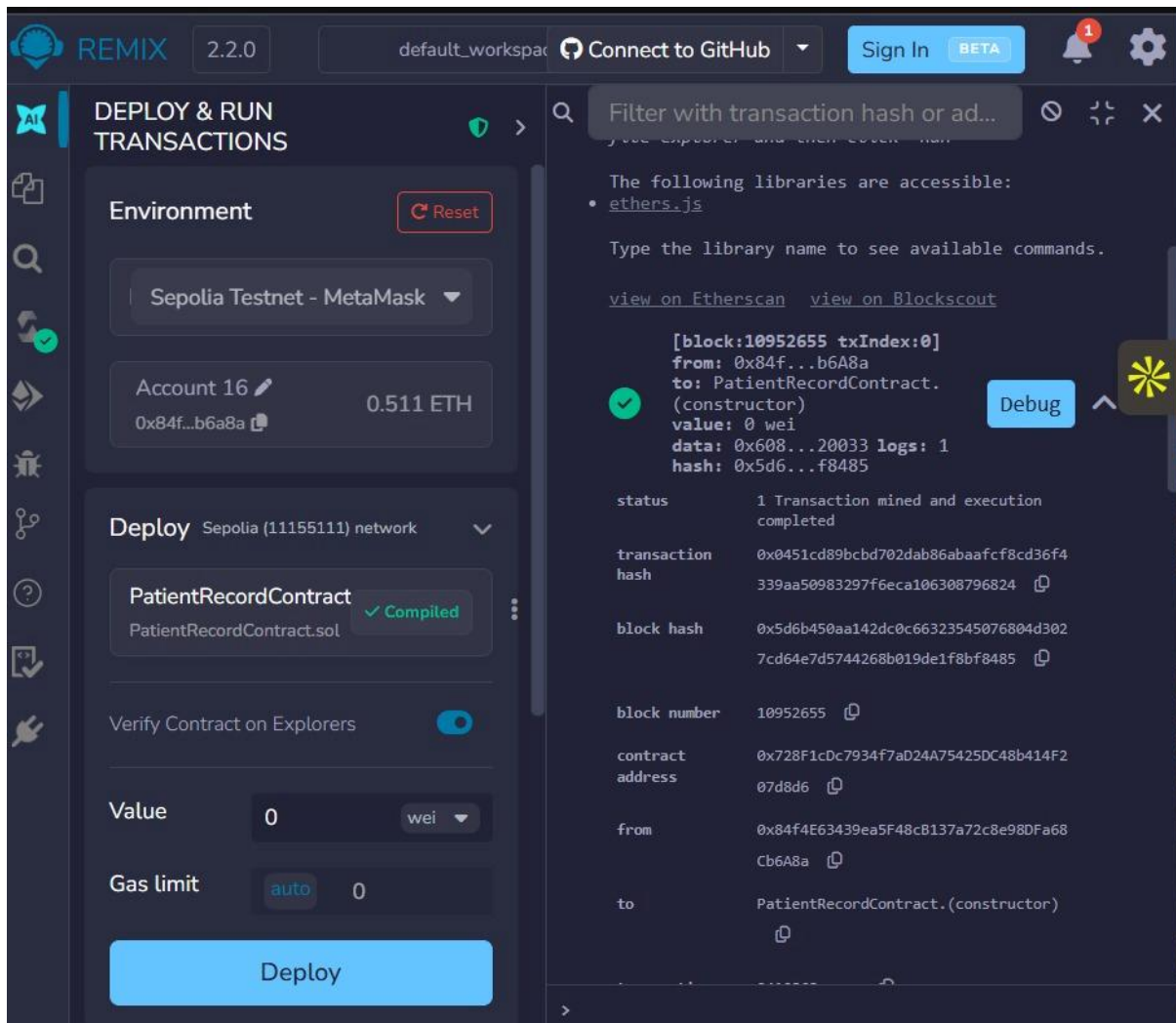


Figure 5.5: Deployment Transaction Confirmation

Transaction confirmation gives cryptographic proof that a deployment was successful, and provides third-party verification by independent actors, or stakeholders (Ethereum Foundation, 2023).

Healthcare systems can use transaction verification to bolster transparency and accountability, giving them a way to independently validate the contract logic deployed

5.6 Smart Contract Verification

Source code verification was performed using:

- Sourceify
- Blockscout


sourceify.eth

[Verify](#)
[sourceify.dev](#)

0x728F1cDc7934f7aD24A75425DC48b414F207d8d6

on Ethereum Sepolia Testnet (11155111)

Exact Match
Runtime Bytecode
Creation Bytecode

Contract Name	PatientRecordContract
Compilation Target	PatientRecordContract.sol:PatientRecordContract
Language	Solidity
Compiler	solc 0.8.34+commit.80d5c536
EVM Version	default
Verified At	2026-05-30 07:15:13 UTC
Deployer	0x84f4E63439ea5F48cB137a72c8e98DFa68Cb6A8a
Deployment Transaction	0x0451cd89bcbd702dab86abaa9cf8cd36f4339aa50983297f6eca106308796824
Block Number	10952655
Transaction Index	0

Figure 5.6: Successful Sourceify Verification

Chains

Gas 15.2 Gwei

Add Sepolia

Sepolia

Blockchain Tokens Dapps Charts & stats API Other

Merits Log in

Launch your own fully functioning blockchain explorer in minutes. [Deploy now](#)

Search by address / txn hash / block / token/...

Sponsored: CoW Swap – Swap Without Paying Gas [Swap Now!](#)

Contract details

0x728F1cDc7934f7aD24A75425DC48b414F207d8d6

95.28 1 Explorer

Details

Contract

Transactions 7

Token transfers 0

Tokens 0

Internal txns 0

Coin balance history

Logs 7

Widgets 3

Contract name

PatientRecordContract

Creator

0x84...6A8a at txn 0x04...6824 Success

Balance

0 ETH

Net worth

N/A

Transactions

0

Gas used

0

Last balance update

10954368

Figure 5.7: Successful Blockscout Verification

Verification ensures that in-processed code is identical with the code originally submitted. It creates trust and transparency as stakeholders can view the logic of the contract and check there is no hidden functionality (Wood, 2022).

In healthcare settings, this transparency is crucial as patient record handling needs to be traceable and reliable.

5.7 Interaction with the Deployed Smart Contract

Following deployment, the contract became accessible through Remix IDE and blockchain explorers.

Several interactions were performed:

- Healthcare record creation.
- Record retrieval.
- Ownership transfer.
- Patient history retrieval.
- Administrative functions.

Each interaction generated a blockchain transaction recorded permanently on the Sepolia network.

The successful execution of these operations confirmed that the deployed contract behaved consistently with its design and correctly enforced access-control rules.

5.8 Evaluation of Deployment

The deployment process was completed successfully without significant technical issues. The combination of Remix IDE, MetaMask, Alchemy, and Sepolia provided a reliable environment for development and testing.

Key benefits observed included:

- Transparent transaction verification.
- Public accessibility through blockchain explorers.
- Immutable contract storage.
- Decentralised execution.

- Independent source-code verification.

However, limitations remain. The contract was deployed on the test network, not the main network of Ethereum, and therefore actual transaction fees and performance under high loads were not tested. Additionally, only healthcare data was recorded on-chain to reduce privacy concerns and storage space (Khatoon, 2020).

6.0 Testing and Evaluation

6.1 Chapter Introduction

The testing phase is an important part of the development of a blockchain system as it ensures the system is functional, secure, consistent and reliable (Mamun, Azam and Gritti, 2022). Testing focused on two goals: the first was to test the Python blockchain implementation, and the second was to test the Ethereum smart contract that was built for HealthCare Innovations Ltd.

Testing involved checking various blockchain operations, smart contract functions, security measures, and healthcare records management.

6.2 Testing Strategy

The testing strategy combined:

- Functional testing.
- Validation testing.
- Security testing.

Testing was performed in two stages:

1. Python Blockchain Testing.
2. Smart Contract Testing on Ethereum Sepolia.

The approach was designed to evaluate all major project requirements, including blockchain creation, mining, consensus, smart contract execution, deployment verification, and healthcare record management.

6.3 Python Blockchain Testing

Test 1: Genesis Block Creation

Objective: Verify successful blockchain initialisation.

Expected Result: Creation of Block 0 with a previous hash consisting of sixty-four zeros.

Actual Result: The genesis block was successfully created.

Outcome: Pass

The successful creation of the genesis block confirmed correct blockchain initialisation.

Test 2: Proof-of-Work Mining

Objective: Verify successful mining of a new block.

Expected Result: Generation of a hash beginning with four leading zeros.

Actual Result: A valid nonce and hash satisfying the difficulty requirement were generated.

Outcome: Pass

The result confirmed correct implementation of the mining algorithm and block validation process (Saleh, 2020).

Test 3: Patient Record Creation

Objective: Verify successful storage of healthcare transactions.

Expected Result: New healthcare records stored as blockchain transactions.

Actual Result: Multiple patient records were successfully created and stored.

Outcome: Pass

The results demonstrated the practical application of blockchain technology for healthcare record management (Shahnaz, Qamar and Khalid, 2019).

Test 4: Blockchain Validation

Objective: Verify blockchain integrity validation.

Expected Result: Validation function confirms ledger integrity.

Actual Result: All blocks remained correctly linked and valid.

Outcome: Pass

This confirms that blockchain integrity mechanisms functioned correctly.

Test 5: Tampering Detection

Objective: Detect unauthorised modification.

Expected Result: Validation failure following data alteration.

Actual Result: The validation algorithm detected the modification and reported an invalid blockchain.

Outcome: Pass

The result demonstrates the effectiveness of blockchain immutability and cryptographic protection mechanisms (Omar et al., 2019).

Test 6: Consensus Mechanism

Objective: Verify longest-chain selection.

Expected Result: Selection of the longest valid blockchain.

Actual Result: The chain containing six blocks was selected successfully.

Outcome: Pass

This confirms successful implementation of decentralised consensus.

6.4 Smart Contract Testing

Test 7: Healthcare Provider Authorisation

Objective: Verify administrative access control.

Expected Result: Successful provider authorisation.

Actual Result: Provider status updated and recorded on-chain.

Outcome: Pass

Cell 5 — Authorise a Second Healthcare Provider

```
14]: # — Second provider address (a separate Ethereum account) —
# This simulates a second healthcare provider on the network.
# Replace with a real second MetaMask account address if available,
# or use this placeholder address for demonstration purposes.
SECOND_PROVIDER = Web3.to_checksum_address("0x742d35cc6634c0532925a3b8d4c9b8e8b6b0e1a2")

print("Authorising second provider...")
print(f" Provider address : {SECOND_PROVIDER}")

# Check if already authorised before sending a transaction
already_authorised = contract.functions.isAuthorised(SECOND_PROVIDER).call()

if already_authorised:
    print(f" Status           : Already authorised – skipping transaction")
else:
    # Build the authoriseProvider transaction
    tx_function = contract.functions.authoriseProvider(SECOND_PROVIDER)
    receipt = send_transaction(tx_function)
    print(f" Transaction hash : {receipt.transactionHash.hex()}")

# Confirm authorisation status
is_authorised = contract.functions.isAuthorised(SECOND_PROVIDER).call()
print(f" Authorised         : {is_authorised}")

Authorising second provider...
Provider address : 0x742d35CC6634c0532925a3B8D4c9b8E8B6b0e1A2
Status           : Already authorised – skipping transaction
Authorised       : True
```

Figure 6.1: Authorise a Second Healthcare Provider

Test 8: Add Patient Records to the Contract

Objective: Add new patient healthcare record to the contract.

Expected Result: Record added to the blockchain.

Actual Result: Record successfully added and recorded on-chain.

Outcome: Pass

Cell 6 — Add Patient Records to the Contract

```
[45]: # — Patient records to add —
records_to_add = [
    {
        "patient_id" : "P001",
        "provider_name" : "Dr. Sarah Smith",
        "hospital" : "City General Hospital",
        "diagnosis_code": "J18.9"
    },
    {
        "patient_id" : "P002",
        "provider_name" : "Dr. James Patel",
        "hospital" : "North Clinic",
        "diagnosis_code": "E11.9"
    },
    {
        "patient_id" : "P003",
        "provider_name" : "Dr. Sarah Smith",
        "hospital" : "City General Hospital",
        "diagnosis_code": "I10"
    }
]

added_record_ids = []
total_to_add = len(records_to_add)
print(f"Adding {total_to_add} patient records to the blockchain...")
print("")

for i, record in enumerate(records_to_add):
    pid = record["patient_id"]
    pname = record["provider_name"]
    pcode = record["diagnosis_code"]
    print(f"[Record {i+1}] Patient: {pid} | Provider: {pname} | Diagnosis: {pcode}")

    timestamp = int(datetime.now().timestamp())
    record_hash = generate_record_hash(pid, pname, pcode, timestamp)

    tx_function = contract.functions.addRecord(
        record["patient_id"],
        record["provider_name"],
        record["hospital"],
        record["diagnosis_code"],
        record_hash
    )

    receipt = send_transaction(tx_function)
    txhash = receipt.transactionHash.hex()
    print(f" Transaction hash : {txhash}")
    print("")

total = contract.functions.totalRecords().call()
print(f"Total records on contract : {total}")
```

Figure 6.2: Adding New Patient Record Creation

Test 9: Record Retrieval

Objective: Retrieve all stored healthcare information.

Expected Result: Accurate record retrieval.

Actual Result: Record data successfully returned.

Outcome: Pass

Cell 7 — Retrieve and Display Records

```
: total = contract.functions.totalRecords().call()
num_records = str(total)
print("Retrieving " + num_records + " records from the contract...")
print("=" * 65)

for record_id in range(1, total + 1):
    record = contract.functions.getRecord(record_id).call()

    print("Record ID      : " + str(record[0]))
    print("Patient ID    : " + str(record[1]))
    print("Provider Name   : " + str(record[2]))
    print("Hospital        : " + str(record[3]))
    print("Diagnosis Code  : " + str(record[4]))
    print("Record Hash     : 0x" + record[5].hex())
    print("Current Provider: " + str(record[6]))
    print("Timestamp       : " + datetime.fromtimestamp(record[7]).strftime('%Y-%m-%d %H:%M:%S'))
    print("Active          : " + str(record[8]))
    print("=" * 65)
```

Figure 6.3: Record Retrieval Function Output

Test 10: Record Ownership Transfer

Objective: Verify ownership transfer.

Expected Result: Record ownership successfully updated.

Actual Result: Ownership transferred and recorded on-chain.

Outcome: Pass

```

[49]: # Transfer Record 3 - still held by admin wallet
      RECORD_ID_TO_TRANSFER = 3

      print("Transferring Record " + str(RECORD_ID_TO_TRANSFER) + " to second provider...")
      print("")

      record_before = contract.functions.getRecord(RECORD_ID_TO_TRANSFER).call()
      print(" Before transfer:")
      print(" Patient ID      : " + str(record_before[1]))
      print(" Current Provider : " + str(record_before[6]))

      tx_function = contract.functions.transferRecord(
          RECORD_ID_TO_TRANSFER,
          SECOND_PROVIDER
      )

      receipt = send_transaction(tx_function)
      print(" Transaction hash : " + receipt.transactionHash.hex())

      record_after = contract.functions.getRecord(RECORD_ID_TO_TRANSFER).call()
      print("")
      print(" After transfer:")
      print(" Patient ID      : " + str(record_after[1]))
      print(" Current Provider : " + str(record_after[6]))
      print("")
      print(" Transfer successful: " + str(record_before[6] != record_after[6]))

Transferring Record 3 to second provider...

Before transfer:
  Patient ID      : P002
  Current Provider : 0x84f4e63439ea5f48c8137a72c8e98dFa68Cb6A8a
Transaction sent  : 2999781a6416e3fd3474e98a8d562f0cef6917ee881f3c2eba24927461976be4
Waiting for mining ...
Status           : SUCCESS
Block number     : 10973940
Gas used         : 39009
Transaction hash  : 2999781a6416e3fd3474e98a8d562f0cef6917ee881f3c2eba24927461976be4

After transfer:
  Patient ID      : P002
  Current Provider : 0x742d35CC6634c0532925a388D4c9b8E886b0e1A2

Transfer successful: True

```

Figure 6.4: Record Transfer Transaction

Test 11: Patient History Retrieval

Objective: Retrieve complete patient history.

Expected Result: Return all patient records.

Actual Result: Patient history successfully returned.

Outcome: Pass

```

Retrieving 10 records from the contract...
=====
Record ID      : 1
Patient ID     : P001
Provider Name  : Dr. Sarah Smith
Hospital      : City General Hospital
Diagnosis Code : J18.9
Record Hash    : 0xd3966ebc3a22609e34a37ac9ab8ad9f7d855067ed67ccf7810796addb1c372a5
Current Provider: 0x742d35cc6634c0532925a388D4c9b8E8B6b0e1A2
Timestamp     : 2026-05-30 13:48:00
Active        : True
=====
Record ID      : 2
Patient ID     : P001
Provider Name  : Dr. Sarah Smith
Hospital      : City General Hospital
Diagnosis Code : J18.9
Record Hash    : 0xccb559940d732899e34349ef387eb7d475f638e15d81b4248e4bd5974f0ec456
Current Provider: 0x84f4E63439ea5F48cB137a72c8e98Dfa68Cb6A8a
Timestamp     : 2026-05-30 13:51:36
Active        : True
=====
Record ID      : 3
Patient ID     : P002
Provider Name  : Dr. James Patel
Hospital      : North Clinic
Diagnosis Code : E11.9
Record Hash    : 0x414b10e995c6236f4e4e56e9114f14cba3f0ab927129f96a076d8bfb68eacfd8
Current Provider: 0x84f4E63439ea5F48cB137a72c8e98Dfa68Cb6A8a
Timestamp     : 2026-05-30 13:51:48
Active        : True

```

Figure 6.5: Patient History Retrieval Output

6.5 Evaluation of Testing Results

All significant system requirements were successfully verified in the testing programme. The implementation of the blockchain successfully confirmed the ability to create blocks and to carry out mining, validation, detection of tampering and consensus. Likewise, the smart contract successfully added the functionalities of managing patients' history, ownership, creation, retrieval, and healthcare providers.

Security-related tests were conducted to ensure that blockchain technology can effectively protect healthcare records from unauthorised modification, maintain transparency and traceability (Agbo et al., 2019; Hasselgren et al., 2020).

The test was done in a controlled setting, but it serves as a solid proof of the possibility of using blockchain in healthcare record management.

7.0 Discussion and Critical Analysis

7.1 Chapter Introduction

This chapter reviews the healthcare record management system developed using blockchain for HealthCare Innovations Ltd. It explores the advantages, challenges, and real-world applications of blockchain in the healthcare industry.

7.2 Strengths of the Proposed Solution

The solution developed showed some benefits of blockchain technology. Firstly, blockchain consisted of cryptographic hashing: data that was modified without the owner's consent would be immediately detected (Zheng et al., 2020; Bodkhe et al., 2020). This is especially crucial in the healthcare industry where precise patient records are crucial.

Secondly, blockchain enhanced transparency and traceability in healthcare, by providing a permanent audit trail of healthcare transactions (Vazirani et al., 2019; Hasselgren et al., 2020). Patient history functionality allowed to track the report ownership and activity.

Thirdly, the solution highlighted the advantages of decentralisation, less relying on centralised databases and increased system resilience (Agbo et al., 2019). Lastly, smart contracts streamlined healthcare operations like patient record generation, ownership, and provider authorisation, making them more efficient and less cumbersome (Khatoon, 2020).

7.3 Limitations of the Proposed Solution

However, some of the following limitations were identified. A major challenge is scalability as blockchain networks are slower at processing transactions than traditional databases (Hussien et al., 2021). Healthcare organisations have a massive amount of data, that could impact performance.

Record storage restrictions are also present as medical records usually include large-sized files like images and reports. This means that only hash of records was recorded on the chains, whereas actual records needed to be stored off chains (Mamun, Azam and Gritti, 2022).

Besides, there are also costs involved in blockchain transactions, especially on the Ethereum network. Security and privacy concerns are also important since health information is sensitive and the immutability of blockchain could pose potential challenges for data protection regulations (Truong et al., 2020; Omar et al., 2019).

7.4 Ethical and Legal Considerations

Patient confidentiality is key for healthcare organisations, and they must adhere to various regulations like GDPR. This was overcome by storing only a cryptographic hash of the patient data, not the actual patient data on the blockchain. In a real world scenario, however, more

security measures such as encryption, identity management and other advanced access-control would need to apply (Al-Zaben et al., 2018).

7.5 Recommendations for Future Development

Some potential future improvements are the integration with decentralized storage solutions like IPFS, the use of more efficient consensus mechanisms like Proof-of-Stake and the interoperability with existing EHR systems (Saleh, 2020; Yaqoob et al., 2021). The system could be further improved by adding more security measures and patient consent management features.

8.0 Conclusion

The goal of this project was to develop a patient record management system (PRMS) for the HealthCare Innovations domain that is based on the Ethereum blockchain and to give an evaluation of the prototype developed. This objective was achieved through a blockchain prototype, the creation of a smart contract in the Solidity programming language, deployment to the Ethereum Sepolia Test Network and the verification of the main system use cases' functionality.

The use of the Python programming language to implement the Proof-of-Concept (PoC) System for the HealthCare Innovations PRMS System showed several core functionality of blockchain technology. These comprise the development of blockchain constructions, the incorporation of cryptographic hash features to guarantee the stability of blocks, tamper proofing, reaching contract and proof of work mining.

The integrated system demonstrated the functionality of the smart contract, and it was also possible to authorize health care providers, to log patient's information, to transfer patient ownership, to query patient's history and to log events in the system.

The combination of the Remix IDE, MetaMask and Alchemy demonstrated that the entire ecosystem was completely incorporated into the Ethereum blockchain system. Verification by Sourceify and BlockScout revealed that the deployed system was completely verifiable. When testing the system, unauthorized changes to the health care records stored by the system were not allowed.

This also highlighted the core advantages of the blockchain system such as decentralization, transparency, traceability and smart contracts automation. In addition to these positive elements, the following difficulties were identified: Compliance with regulations, the slow throughput of blockchain transactions and the cost of transactions.

References

- AbdelSalam, F.M. (2023) 'Blockchain revolutionizing healthcare industry: A systematic review of blockchain technology benefits and threats', *Perspectives in Health Information Management*, 20(3), p. 1b. Available at: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10701638/> (Accessed: 1 June 2026).
- Agbo, C.C., Mahmoud, Q.H. and Eklund, J.M. (2019) 'Blockchain technology in healthcare: A systematic review', *Healthcare*, 7(2), p. 56. Available at: <https://doi.org/10.3390/healthcare7020056>.
- Al-Zaben, N., Hassan Onik, M.M., Yang, J., Lee, N.-Y. and Kim, C.-S. (2018) 'General Data Protection Regulation complied blockchain architecture for personally identifiable information management', in *2018 International Conference on Computing, Electronics & Communications Engineering (iCCECE)*. Southend, UK: IEEE, pp. 151–156. Available at: <https://doi.org/10.1109/ICCECOME.2018.8658586>.
- Angraal, S., Krumholz, H.M. and Schulz, W.L. (2017) 'Blockchain technology: Applications in health care', *Circulation: Cardiovascular Quality and Outcomes*, 10(9), e003800. Available at: <https://doi.org/10.1161/CIRCOUTCOMES.117.003800>.
- Bodkhe, U., Tanwar, S., Parekh, K., Khanpara, P., Tyagi, S., Kumar, N. and Alazab, M. (2020) 'Blockchain for Industry 4.0: A comprehensive review', *IEEE Access*, 8, pp. 79764–79800. Available at: <https://doi.org/10.1109/ACCESS.2020.2988579>.
- Buterin, V. (2014) *A Next Generation Smart Contract and Decentralized Application Platform*. Available at: https://blockchainlab.com/pdf/Ethereum_white_paper-a_next_generation_smart_contract_and_decentralized_application_platform-vitalik-buterin.pdf (Accessed: 1 June 2026).
- Culot, G., Fattori, F., Podrecca, M. and Sartor, M. (2019) 'Addressing Industry 4.0 cybersecurity challenges', *IEEE Engineering Management Review*, 47(3), pp. 79–86. Available at: <https://doi.org/10.1109/EMR.2019.2927559>.
- Dang, L.M., Piran, M.J., Han, D., Min, K. and Moon, H. (2019) 'A survey on Internet of Things and cloud computing for healthcare', *Electronics*, 8(7), p. 768. Available at: <https://doi.org/10.3390/electronics8070768>.
- Di Francesco Maesa, D. and Mori, P. (2020) 'Blockchain 3.0 applications survey', *Journal of Parallel and Distributed Computing*, 138, pp. 99–114. Available at: <https://doi.org/10.1016/j.jpdc.2019.12.019>.
- Dwivedi, Y.K. et al. (2022) 'Metaverse beyond the hype: Multidisciplinary perspectives on emerging challenges, opportunities, and agenda for research, practice and policy', *International Journal of Information Management*, 66, 102542. Available at: <https://doi.org/10.1016/j.ijinfomgt.2022.102542>.

El-Gazzar, R. and Stendal, K. (2020) 'Blockchain in health care: Hope or hype?', *Journal of Medical Internet Research*, 22(7), e17199. Available at: <https://doi.org/10.2196/17199>.

Ethereum Foundation (2023) *Ethereum Development Documentation*. Available at: <https://ethereum.org/developers/docs/> (Accessed: 1 June 2026).

Exceline, C.E. and Nagarajan, S. (2023) 'The security of blockchain-based electronic health record: A systematic review', *International Journal of Computer Network and Information Security*, 15(6), pp. 103–118. Available at: <https://www.mecspress.org/ijcnis/ijcnis-v15-n6/v15n6-9.html> (Accessed: 1 June 2026).

Haleem, A., Javaid, M., Singh, R.P., Suman, R. and Rab, S. (2021) 'Blockchain technology applications in healthcare: An overview', *International Journal of Intelligent Networks*, 2, pp. 130–139. Available at: <https://doi.org/10.1016/j.ijin.2021.09.005>.

Hasselgren, A., Krlevska, K., Gligoroski, D., Pedersen, S.A. and Faxvaag, A. (2020) 'Blockchain in healthcare and health sciences—A scoping review', *International Journal of Medical Informatics*, 134, 104040. Available at: <https://doi.org/10.1016/j.ijmedinf.2019.104040>.

Hussien, H.M., Yasin, S.M., Udzir, N.I., Ninggal, M.I.H. and Salman, S. (2021) 'Blockchain technology in the healthcare industry: Trends and opportunities', *Journal of Industrial Information Integration*, 22, 100217. Available at: <https://doi.org/10.1016/j.jii.2021.100217>.

Ismail, L., Materwala, H. and Zeadally, S. (2019) 'Lightweight blockchain for healthcare', *IEEE Access*, 7, pp. 149935–149951. Available at: <https://doi.org/10.1109/ACCESS.2019.2947613>.

Khatoon, A. (2020) 'A blockchain-based smart contract system for healthcare management', *Electronics*, 9(1), p. 94. Available at: <https://doi.org/10.3390/electronics9010094>.

Krittanawong, C., Rogers, A.J., Aydar, M., Choi, E., Johnson, K.W., Wang, Z. and Narayan, S.M. (2020) 'Integrating blockchain technology with artificial intelligence for cardiovascular medicine', *Nature Reviews Cardiology*, 17(1), pp. 1–3. Available at: <https://doi.org/10.1038/s41569-019-0294-y>.

Mamun, A.A., Azam, S. and Gritti, C. (2022) 'Blockchain-based electronic health records management: A comprehensive review and future research direction', *IEEE Access*, 10, pp. 1–24. Available at: <https://doi.org/10.1109/ACCESS.2022.3141079>.

McGhin, T., Choo, K.-K.R., Liu, C.Z. and He, D. (2019) 'Blockchain in healthcare applications: Research challenges and opportunities', *Journal of Network and Computer Applications*, 135, pp. 62–75. Available at: <https://doi.org/10.1016/j.jnca.2019.02.027>.

Mearian, L. (2025) *The Merge*. Available at: <https://ethereum.org/roadmap/merge/> (Accessed: 1 June 2026).

Nakamoto, S. (2008) *Bitcoin: A Peer-to-Peer Electronic Cash System*. Available at: <https://bitcoin.org/bitcoin.pdf> (Accessed: 1 June 2026).

Naresh, V.S., Sada, R., Allu, R.J.P., Gubbala, A.D. and Bandaru, U.D. (2025) 'Exploring the potential of blockchain technology in modern healthcare systems', *Peer-to-Peer Networking and Applications*, 18(6), pp. 1–18. Available at: <https://doi.org/10.1007/s12083-025-02132-3>.

Ohvanainen, A., Niemi-Murola, L., Elonheimo, O. and Pöyhiä, R. (2021) 'Hospital-at-home network in Finland', *International Journal of Healthcare Management*, 14(4), pp. 1018–1024. Available at: <https://doi.org/10.1080/20479700.2020.1725717>.

Omar, A.A., Bhuiyan, M.Z.A., Basu, A., Kiyomoto, S. and Rahman, M.S. (2019) 'Privacy-friendly platform for healthcare data in cloud based on blockchain environment', *Future Generation Computer Systems*, 95, pp. 511–521. Available at: <https://doi.org/10.1016/j.future.2018.12.044>.

Patel, V. (2019) 'A framework for secure and decentralized sharing of medical imaging data via blockchain consensus', *Health Informatics Journal*, 25(4), pp. 1398–1411. Available at: <https://doi.org/10.1177/1460458218769699>.

Politou, E., Casino, F., Alepis, E. and Patsakis, C. (2019) 'Blockchain mutability: Challenges and proposed solutions', *IEEE Transactions on Emerging Topics in Computing*, 9(4), pp. 1972–1986. Available at: <https://doi.org/10.1109/TETC.2019.2949510>.

Saleh, F. (2021) 'Blockchain without waste: Proof-of-Stake', *Review of Financial Studies*, 34(3), pp. 1156–1190. Available at: <https://doi.org/10.1093/rfs/hhaa075>.

Saranya, R. and Murugan, A. (2021) 'A systematic review of enabling blockchain in healthcare system: Analysis, current status, challenges and future direction', *Materials Today: Proceedings*, 47, pp. 2094–2101. Available at: <https://doi.org/10.1016/j.matpr.2021.07.105>.

Sethi, N., Rathore, C. and Singh, D. (2025) 'Blockchain as a prime guardian: Securing clinical trial data integrity', *Reviews on Recent Clinical Trials*, 20(1), pp. 1–12. Available at: <https://doi.org/10.2174/0115748871357146250116113310>.

Shahnaz, A., Qamar, U. and Khalid, A. (2019) 'Using blockchain for electronic health records', *IEEE Access*, 7, pp. 147782–147795. Available at: <https://doi.org/10.1109/ACCESS.2019.2946373>.

Shaikh, M., Memon, S.A., Ebrahimi, A. and Wiil, U.K. (2025) 'A systematic literature review for blockchain-based healthcare implementations', *Healthcare*, 13(9), p. 1087. Available at: <https://doi.org/10.3390/healthcare13091087>.

Singh, S., Sharma, S.K., Mehrotra, P., Bhatt, P. and Kaurav, M. (2022) 'Blockchain technology for efficient data management in healthcare system: Opportunity, challenges and future

perspectives', *Materials Today: Proceedings*, 62, pp. 5042–5048. Available at: <https://doi.org/10.1016/j.matpr.2022.04.998>.

Siyal, A., Junejo, A.Z., Zawish, M., Ahmed, K., Khalil, A. and Soursou, G. (2019) 'Applications of blockchain technology in medicine and healthcare: Challenges and future perspectives', *Cryptography*, 3(1), p. 3. Available at: <https://doi.org/10.3390/cryptography3010003>.

Szabo, N. (1997) *Formalizing and Securing Relationships on Public Networks*. Available at: <https://www.fon.hum.uva.nl/rob/Courses/InformationInSpeech/CDROM/Literature/LOTwinterschool2006/szabo.best.vwh.net/formalize.html> (Accessed: 1 June 2026).

Tandon, A., Dhir, A., Islam, N. and Mäntymäki, M. (2020) 'Blockchain in healthcare: A systematic literature review, synthesizing framework and future research agenda', *Computers in Industry*, 122, 103290. Available at: <https://doi.org/10.1016/j.compind.2020.103290>.

Tith, D., Lee, J.-S., Suzuki, H., Wijesundara, W.M.A.B., Taira, N., Obi, T. and Ohyama, N. (2020) 'Patient consent management by a purpose-based consent model for electronic health record based on blockchain technology', *Healthcare Informatics Research*, 26(4), pp. 265–273. Available at: <https://doi.org/10.4258/hir.2020.26.4.265>.

Truong, N.B., Sun, K., Lee, G.M. and Guo, Y. (2020) 'GDPR-compliant personal data management: A blockchain-based solution', *IEEE Transactions on Information Forensics and Security*, 15, pp. 1746–1761. Available at: <https://doi.org/10.1109/TIFS.2019.2948287>.

Vazirani, A.A., O'Donoghue, O., Brindley, D. and Meinert, E. (2019) 'Implementing blockchains for efficient health care: Systematic review', *Journal of Medical Internet Research*, 21(2), e12439. Available at: <https://doi.org/10.2196/12439>.

Wood, G. (2022) *Ethereum: A Secure Decentralised Generalised Transaction Ledger*. Available at: <https://ethereum.github.io/yellowpaper/paper.pdf> (Accessed: 1 June 2026).

Yaqoob, I., Salah, K., Jayaraman, R. and Al-Hammadi, Y. (2022) 'Blockchain for healthcare data management: Opportunities, challenges, and future recommendations', *Neural Computing and Applications*, 34(14), pp. 11475–11490. Available at: <https://doi.org/10.1007/s00521-020-05519-w>.

Zheng, Z., Xie, S., Dai, H.-N., Chen, W., Chen, X., Weng, J. and Imran, M. (2020) 'An overview on smart contracts: Challenges, advances and platforms', *Future Generation Computer Systems*, 105, pp. 475–491. Available at: <https://doi.org/10.1016/j.future.2019.12.019>

Appendices

Appendix 1: Gantt Chart showing the timeline for project completion

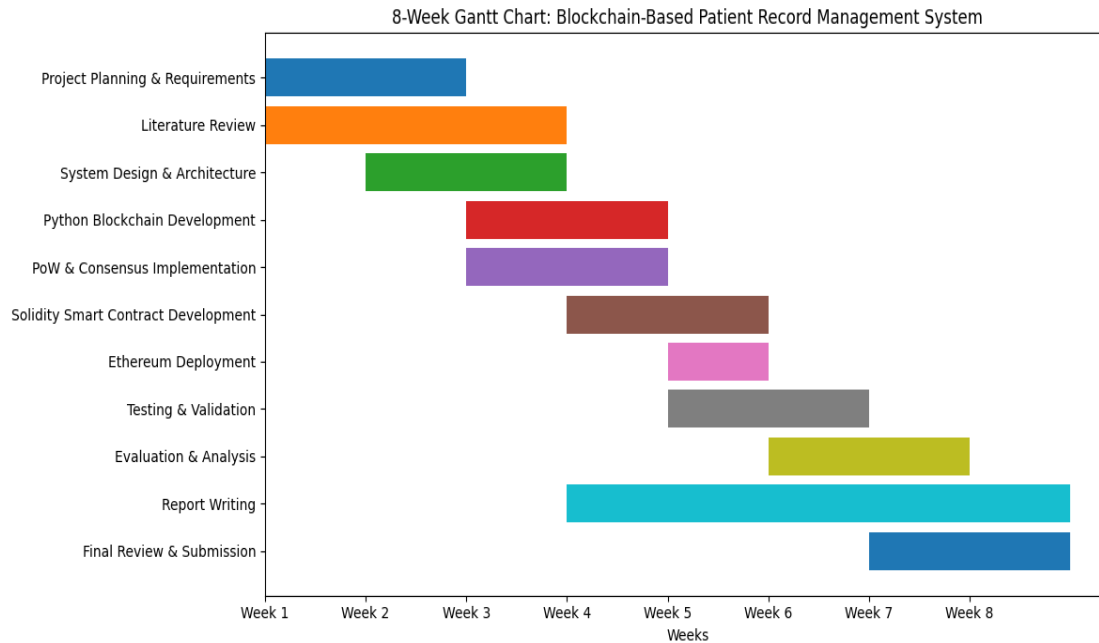


Figure A.1: Gantt Chart Showing the Eight-Week Project Schedule for the Development of a Blockchain-Based Patient Record Management System

Appendix 2: Development Environment Configuration

Component	Technology Used
Programming Language	Python 3.x
Smart Contract Language	Solidity
Blockchain Platform	Ethereum
Blockchain Network	Sepolia Test Network
Development IDE	Remix IDE
Cryptocurrency Wallet	MetaMask
Blockchain Infrastructure Provider	Alchemy

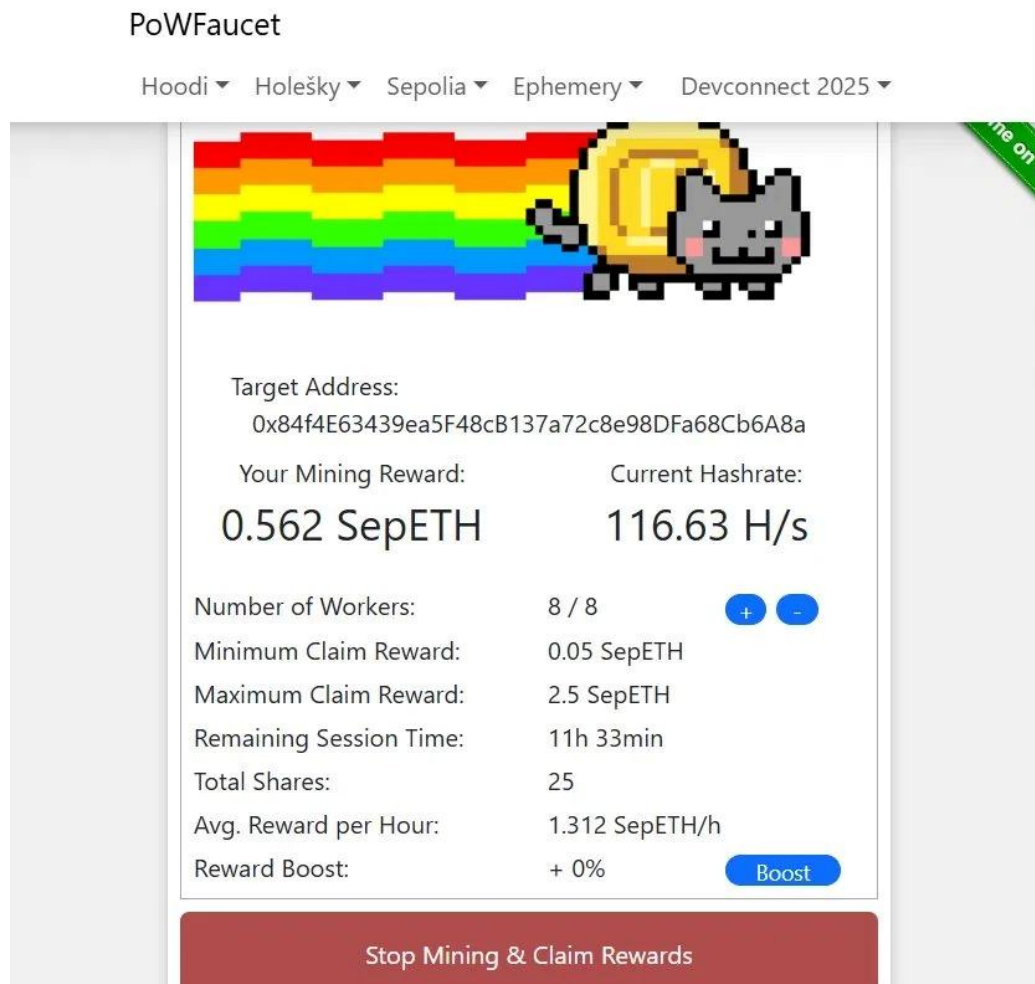
Smart Contract Verification Tools	Sourceify, Blockscout
-----------------------------------	-----------------------

Appendix 3: Python Blockchain Source Code

Figure A.3: Complete Python implementation

Appendix 4: Solidity Smart Contract Source Code

Figure A.4: Complete PatientRecordContract.sol implementation



Coin mining for contract deployment

Sepolia PoW Faucet

Claim Rewards

Wallet: 0x84f4E63439ea5F48cB137a72c8e98DFa68Cb6A8a
Amount: 0.563 SepETH
Timeout: -

Claim Transaction has been confirmed in block #10952642!

TX: [0x8f49f69caa30b62a7ccde4a8663242a8177cae0c1a04a47d1dcc0e2fd71e1d67](https://etherscan.io/tx/0x8f49f69caa30b62a7ccde4a8663242a8177cae0c1a04a47d1dcc0e2fd71e1d67)

Did you like the faucet? Give that project a  Star 5,515

Or support this faucet by sharing your result with a  Tweet 

Post

Return to
startpage

Coin successfully mined and transfered to Metamask wallet

